МЕТОДИЧЕСКОЕ ПОСОБИЕ

Программирование на Python

Справочник для учеников: все темы курса с объяснениями и примерами — от первой программы до игр на Pygame и чат-ботов

0 Старт
1 Основы Python
2 Строки и коллекции
3 Функции и модули
4 Алгоритмы
5 Файлы и данные
6 ООП
7 Pygame
8 Python для реальных задач
+ Шпаргалки

ДТ «Кванториум» 2026

Содержание

Как пользоваться методичкой 4

Старт 5

0.1. Что такое программа и Python 5

0.2. Установка и первая программа 5

0.3. Git и GitHub 7

Часть 1. Основы Python 9

1.1. Переменные 9

1.2. Типы данных 10

1.3. Ввод и вывод 12

1.4. Арифметика 14

1.5. Условия: if, elif, else 16

1.6. Логические операторы и match-case 18

1.7. Цикл for и range 20

1.8. Цикл while, break и continue 22

1.9. Вложенные циклы и случайные числа 24

1.10. Ошибки и отладка 25

Часть 2. Строки и коллекции 28

2.1. Строки: индексы и срезы 28

2.2. Методы строк 30

2.3. Списки 32

2.4. Обработка списков 35

2.5. Двумерные списки 37

2.6. Кортежи 38

2.7. Словари 39

2.8. Множества 42

2.9. Какую коллекцию выбрать 43

Часть 3. Функции и модули 44

3.1. Функции 44

3.2. Параметры функций подробнее 47

3.3. Область видимости 48

3.4. Обработка ошибок: try и except 49

3.5. Стандартная библиотека 52

3.6. Свои модули 53

3.7. pip и виртуальные окружения 54

3.8. Функции как значения и lambda (доп.) 55

Часть 4. Алгоритмы 58

4.1. Сложность алгоритмов 58

4.2. Поиск 59

4.3. Простые сортировки 60

4.4. Рекурсия 62

4.5. Быстрые сортировки 64

4.6. Стек, очередь и поиск пути (доп.) 66

Часть 5. Файлы и данные 68
5.1. Текстовые файлы 68
5.2. Пути и папки 69
5.3. CSV: таблицы 70
5.4. JSON 71
5.5. Дата и время 73
5.6. Графики в matplotlib (доп.) 75

Часть 6. ООП 77
6.1. Классы и объекты 77
6.2. Взаимодействие объектов 78
6.3. Инкапсуляция и свойства 79
6.4. Атрибуты класса, staticmethod и classmethod 81
6.5. Магические методы 83
6.6. Наследование 85
6.7. Полиморфизм и абстрактные классы 88
6.8. Композиция и dataclass 89

Часть 7. Pygame 92
7.1. Окно и игровой цикл 92
7.2. Рисование фигур 93
7.3. Движение и управление с клавиатуры 95
7.4. Мышь и текст 99
7.5. Картинки и звук 100
7.6. Спрайты и группы 101
7.7. Столкновения 102
7.8. Состояния игры 104
7.9. Прыжки и гравитация (доп.) 108

Часть 8. Python для реальных задач 111
8.1. Данные из интернета 111
8.2. Базы данных: SQLite 113
8.3. Чат-боты 115
8.4. Оконные приложения на tkinter (доп.) 117

Шпаргалки 119
Встроенные функции 119
Методы строк 120
Методы списков и словарей 120
Частые ошибки 121
Горячие клавиши VS Code 122

Что дальше 123
Где учиться и искать ответы 123

Как пользоваться методичкой

Это справочник для тех, кто учится программировать на Python. Открывай его, когда забыл тему, хочешь повторить перед проектом или интересно заглянуть вперёд.

Темы идут в том же порядке, что и на занятиях: от первой программы до своих игр и чат-ботов. Каждая тема — это объяснение простыми словами и примеры кода. Все примеры можно и нужно запускать у себя: так понять получается намного быстрее, чем просто читая.

Код в методичке выглядит так:

```python
print("Привет, мир!")
```

PYTHON

А то, что программа выведет на экран, — так:

> РЕЗУЛЬТАТ
>
> ```
> Привет, мир!
> ```

Если программа что-то спрашивает у пользователя, то, что напечатал человек, в результате подчёркнуто:

> РЕЗУЛЬТАТ
>
> ```
> Как тебя зовут? Аня
> Привет, Аня!
> ```

Ещё в тексте встречаются заметки:

> **Важно.** Частые ошибки и вещи, без которых код не будет работать.
>
> **Совет.** Приёмы, которые экономят время и нервы.
>
> **Запомни.** Самое главное из темы в одной-двух строчках.

Темы с пометкой «ДОП. ТЕМА» — дополнительные. Их можно пропустить, но если интересно — читай.

НАЧАЛО

Старт

Прежде чем писать код, разберёмся, что такое программа, как Python её выполняет и где писать код.

0.1 Что такое программа и Python

Программа — это список команд для компьютера. Компьютер очень быстрый, но совсем не догадливый: он выполняет команды *ровно* так, как написано, одну за другой. Если в команде ошибка, он не поймёт, что ты имел в виду.

Компьютер понимает только машинный код — длинные последовательности нулей и единиц. Писать так людям неудобно, поэтому придумали **языки программирования**: на них команды пишутся почти обычными словами, а специальная программа переводит их для компьютера.

Python — один из самых популярных языков в мире. Его любят за простой и понятный синтаксис: программы на Python читаются почти как текст на английском.

На Python пишут:

- игры (например, с библиотекой Pygame);
- сайты и серверы;
- чат-ботов;
- программы для анализа данных и нейросети;
- скрипты, которые автоматизируют скучную работу: переименовать 500 файлов, собрать отчёт, скачать данные.

Python — **интерпретируемый** язык. Это значит, что программа-интерпретатор читает твой код строка за строкой и сразу выполняет каждую команду. Поэтому, если в 10-й строке ошибка, первые 9 строк всё равно успеют выполниться.

0.2 Установка и первая программа

Что нужно установить

1. **Python** — с официального сайта python.org. При установке на Windows обязательно поставь галочку **«Add Python to PATH»**: без неё Python не будет запускаться из терминала.
2. **Редактор кода** — VS Code с расширением Python. Для самого первого знакомства подойдёт и простой редактор Thonny.

Проверить, что Python установлен, можно в терминале:

ТЕРМИНАЛ

```
python --version
```

РЕЗУЛЬТАТ

```
Python 3.12.4
```

> **Совет.** На macOS и Linux команда может называться `python3`, а не `python`.

Первая программа

Создай файл `hello.py` (у файлов с кодом на Python расширение `.py`) и напиши в нём:

PYTHON

```python
print("Привет, мир!")
print("Я учу Python")
```

Запусти файл кнопкой ▶ в редакторе или командой в терминале:

ТЕРМИНАЛ

```
python hello.py
```

РЕЗУЛЬТАТ

```
Привет, мир!
Я учу Python
```

`print()` — это **функция**, которая выводит на экран то, что написано в скобках. Текст обязательно берётся в кавычки: одинарные `'...'` или двойные `"..."` — без разницы, главное, чтобы открывающая и закрывающая совпадали.

`print()` умеет выводить и числа, и результаты вычислений:

PYTHON

```python
print(2 + 3)
print("2 + 3")
```

РЕЗУЛЬТАТ

```
5
2 + 3
```

Видишь разницу? Без кавычек Python считает выражение, а в кавычках — просто печатает текст как есть.

Комментарии

Всё, что написано после символа `#`, Python пропускает. Это **комментарии** — заметки для людей, которые читают код.

```python
# Эта программа здоровается с пользователем
print("Привет!")  # комментарий может стоять и в конце строки
```

РЕЗУЛЬТАТ

```
Привет!
```

Интерактивный режим

Если в терминале просто набрать `python` (без имени файла), откроется **интерактивный режим**: пишешь команду — сразу видишь результат. Удобно, чтобы быстро что-то проверить или посчитать. Строки начинаются с `>>>`.

```
>>> 2 ** 100
1267650600228229401496703205376
>>> 60 * 60 * 24 * 365
31536000
>>> exit()
```

0.3 Git и GitHub

Представь, что ты пишешь игру и случайно удалил половину кода, а потом сохранил файл. Без Git код пропал. С Git — можно вернуться к любой сохранённой версии.

Git — это программа, которая хранит историю изменений проекта, как точки сохранения в игре. **GitHub** — сайт, где проекты хранятся в интернете: их можно показать другим и скачать на любом компьютере.

Главные слова:

- **репозиторий** — папка проекта, за которой следит Git;
- **коммит** — сохранённый «снимок» проекта с подписью, что изменилось.

Основные команды (пишутся в терминале, в папке проекта):

ТЕРМИНАЛ

```
git init                        # начать следить за папкой (один раз)
git add .                       # отметить все изменённые файлы
git commit -m "Добавил меню"    # сохранить снимок с подписью
git log --oneline               # посмотреть историю
git push                        # отправить коммиты на GitHub
```

Обычная работа выглядит так: написал кусок кода → проверил, что работает → `git add .` → `git commit -m "что сделал"`. И так много раз за занятие.

> **Совет.** Подпись коммита должна объяснять, что изменилось: «Добавил подсчёт очков», а не «fix» или «123». Через месяц ты скажешь себе спасибо.

Файл .gitignore

Некоторые файлы в Git добавлять не нужно: служебные папки Python и виртуального окружения. Создай в проекте файл `.gitignore` и напиши в нём:

```
__pycache__/
.venv/
.env
```

README

На GitHub в каждом проекте должен быть файл `README.md` — описание: что это за программа, как её запустить, скриншот. Это первое, что увидит человек, открывший твой проект.

```
# Виселица

Консольная игра «Виселица» на Python.
Слова разбиты по категориям: животные, города, фрукты.

Запуск: python main.py
```

ЧАСТЬ 1

Основы Python

Переменные, типы данных, ввод и вывод, условия и циклы — это фундамент любой программы. Из этих кирпичиков собирается всё остальное, от калькулятора до игры.

1.1 Переменные

Переменная — это имя, под которым программа запоминает значение. Можно представить коробку с наклейкой: на наклейке имя, внутри — значение.

```python
name = "Аня"
age = 14
print(name)
print(age)
```

РЕЗУЛЬТАТ

```
Аня
14
```

Знак `=` в программировании означает не «равно», а **«положить в переменную»**. Строку `age = 14` читают так: «переменной `age` присвоить значение 14».

Переменную можно менять

```python
score = 0
print(score)
score = 10
print(score)
score = score + 5    # взять старое значение, прибавить 5, положить обратно
print(score)
```

РЕЗУЛЬТАТ

```
0
10
15
```

Строка `score = score + 5` сначала вычисляет правую часть (`10 + 5` = `15`), а потом кладёт результат обратно в переменную.

Несколько переменных сразу

```python
x, y = 10, 20
print(x, y)

x, y = y, x          # поменять значения местами
print(x, y)
```

РЕЗУЛЬТАТ

```
10 20
20 10
```

Как называть переменные

- Имя может состоять из латинских букв, цифр и знака `_`, но не может начинаться с цифры: `player1` — можно, `1player` — нельзя.
- Большие и маленькие буквы различаются: `Score` и `score` — разные переменные.
- Если имя из нескольких слов, их соединяют подчёркиванием: `player_name`, `max_score`. Такой стиль называется `snake_case`.
- Имя должно объяснять, что внутри: `lives` понятнее, чем `l` или `x1`.

```python
player_name = "Rex"   # хорошо
max_score = 100       # хорошо
a = 100               # плохо: непонятно, что это
```

Совет. Значения, которые не меняются во время работы программы, принято называть заглавными буквами: `MAX_LIVES = 3`, `SCREEN_WIDTH = 800`. Так сразу видно, что это настройка, а не обычная переменная.

Запомни. `=` — это «положить значение в переменную». Справа вычисляется, слева — куда положить.

1.2 Типы данных

У каждого значения в Python есть **тип**. Тип определяет, что это за данные и что с ними можно делать.

Тип	Что хранит	Примеры
`int`	целые числа	`5`, `-12`, `0`, `1000000`
`float`	дробные числа	`3.14`, `-0.5`, `2.0`
`str`	строки (текст)	`"Привет"`, `'Python'`, `"123"`
`bool`	логическое значение: да или нет	`True`, `False`
`None`	«ничего», пустое значение	`None`

Узнать тип значения можно функцией `type()`:

```python
print(type(42))
print(type(3.14))
print(type("Привет"))
print(type(True))
```

РЕЗУЛЬТАТ

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

> **Важно.** Дробную часть отделяют **точкой**, а не запятой: `3.14`, а не `3,14`.

`"5"` и `5` — это разные вещи

```python
print(5 + 5)
print("5" + "5")
```

РЕЗУЛЬТАТ

```
10
55
```

Числа складываются, а строки — склеиваются. `"5"` в кавычках — это текст из одного символа «5», а не число.

Тип можно менять

Python сам определяет тип по значению. Если положить в переменную значение другого типа, тип переменной тоже поменяется:

```python
x = 10
print(type(x))
x = "десять"
print(type(x))
```

РЕЗУЛЬТАТ

```
<class 'int'>
<class 'str'>
```

Странности дробных чисел

```python
print(0.1 + 0.2)
```

РЕЗУЛЬТАТ

```
0.30000000000000004
```

Это не ошибка Python, так работают дробные числа во всех языках: компьютер хранит их с крошечной неточностью. Для вывода такие числа округляют (как — увидишь в следующей теме).

1.3 Ввод и вывод

print(): вывод на экран

В `print()` можно передать несколько значений через запятую — они выведутся через пробел:

```python
name = "Аня"
age = 14
print("Меня зовут", name, "и мне", age, "лет")
```

РЕЗУЛЬТАТ

```
Меня зовут Аня и мне 14 лет
```

У `print()` есть два полезных параметра:

- `sep` — чем разделять значения (по умолчанию пробел);
- `end` — что поставить в конце (по умолчанию переход на новую строку).

```python
print("12", "05", "2026", sep=".")
print("Загрузка", end="...")
print("готово!")
```

РЕЗУЛЬТАТ

```
12.05.2026
Загрузка...готово!
```

f-строки: вставляем значения в текст

Самый удобный способ собрать текст из переменных — **f-строка**. Перед кавычками ставится буква `f`, а переменные пишутся в фигурных скобках:

```python
name = "Аня"
age = 14
print(f"Меня зовут {name}, мне {age} лет")
print(f"Через 5 лет мне будет {age + 5}")
```

РЕЗУЛЬТАТ

```
Меня зовут Аня, мне 14 лет
Через 5 лет мне будет 19
```

Внутри фигурных скобок можно писать любое выражение. А ещё — указывать формат числа:

```python
price = 1234.5678
print(f"Цена: {price:.2f} ₽")     # 2 знака после точки
print(f"Население: {12655050:,}")  # разделитель тысяч
```

РЕЗУЛЬТАТ

```
Цена: 1234.57 ₽
Население: 12,655,050
```

input(): ввод с клавиатуры

`input()` останавливает программу и ждёт, пока пользователь что-нибудь напечатает и нажмёт Enter. Текст в скобках — подсказка, которую увидит пользователь.

```python
name = input("Как тебя зовут? ")
print(f"Привет, {name}!")
```

РЕЗУЛЬТАТ

```
Как тебя зовут? Аня
Привет, Аня!
```

> **Важно.** `input()` **всегда** возвращает строку, даже если пользователь ввёл число.

```python
age = input("Сколько тебе лет? ")
print(age + 1)
```

РЕЗУЛЬТАТ

```
Сколько тебе лет? 14
TypeError: can only concatenate str (not "int") to str
```

Python не может сложить строку `"14"` и число `1`. Сначала строку нужно **превратить в число**.

Преобразование типов

Функция	Что делает	Пример
`int()`	в целое число	`int("14")` → `14`
`float()`	в дробное число	`float("3.5")` → `3.5`
`str()`	в строку	`str(14)` → `"14"`

```python
age = int(input("Сколько тебе лет? "))
print(f"Через год тебе будет {age + 1}")
```

РЕЗУЛЬТАТ

```
Сколько тебе лет? 14
Через год тебе будет 15
```

А если пользователь введёт не число, а, например, слово «десять»? Программа упадёт с ошибкой `ValueError`. Пока что можно проверить строку методом `.isdigit()` — он возвращает `True`, если строка состоит только из цифр:

```python
text = input("Введи число: ")
if text.isdigit():
    print(int(text) * 2)
else:
    print("Это не число!")
```

РЕЗУЛЬТАТ

```
Введи число: десять
Это не число!
```

Как защищать программу от любых ошибок ввода, разберём в теме 3.4.

> **Запомни.** `input()` возвращает строку. Нужно число — оберни в `int()` или `float()`.

1.4 Арифметика

Оператор	Что делает	Пример	Результат
`+`	сложение	`7 + 2`	`9`
`-`	вычитание	`7 - 2`	`5`
`*`	умножение	`7 * 2`	`14`
`/`	деление	`7 / 2`	`3.5`
`//`	целочисленное деление	`7 // 2`	`3`
`%`	остаток от деления	`7 % 2`	`1`
`**`	возведение в степень	`7 ** 2`	`49`

Обычное деление `/` всегда даёт дробное число, даже если делится нацело: `10 / 2` → `5.0`.

Зачем нужны // и %

Целочисленное деление `//` отвечает на вопрос «сколько целых раз помещается», а остаток `%` — «сколько осталось».

Пример: 135 минут — это сколько часов и минут?

```python
minutes = 135
hours = minutes // 60
rest = minutes % 60
print(f"{hours} ч {rest} мин")
```

РЕЗУЛЬТАТ

```
2 ч 15 мин
```

Остаток от деления очень часто используют, чтобы проверить чётность: если `x % 2 == 0`, число чётное. А последняя цифра числа — это `x % 10`.

```python
number = 2026
print(number % 10)    # последняя цифра
print(number // 10)   # число без последней цифры
```

РЕЗУЛЬТАТ

```
6
202
```

Порядок действий

Как в математике: сначала `**`, потом `*`, `/`, `//`, `%`, потом `+` и `-`. Скобки меняют порядок.

```python
print(2 + 3 * 4)
print((2 + 3) * 4)
```

РЕЗУЛЬТАТ

```
14
20
```

Сокращённая запись

```python
score = 10
score += 5     # то же, что score = score + 5
score -= 2     # score = score - 2
score *= 3     # score = score * 3
print(score)
```

РЕЗУЛЬТАТ

```
39
```

Полезные функции

```python
print(abs(-7))            # модуль числа
print(round(3.14159, 2))  # округление до 2 знаков
print(max(4, 9, 2))       # наибольшее
print(min(4, 9, 2))       # наименьшее
```

РЕЗУЛЬТАТ

```
7
3.14
9
2
```

Для более сложной математики есть **модуль** `math`. Его нужно подключить командой `import`:

```python
import math

print(math.sqrt(16))   # квадратный корень
print(math.pi)         # число пи
print(math.ceil(4.1))  # округление вверх
print(math.floor(4.9)) # округление вниз
```

РЕЗУЛЬТАТ

```
4.0
3.141592653589793
5
4
```

Совет. В Python целые числа не переполняются. Можно посчитать `2 ** 1000`, и Python спокойно выведет число из 302 цифр.

1.5 Условия: if, elif, else

Программа часто должна выбирать, что делать: если пароль верный — пустить, иначе — нет. Для этого есть **условный оператор** `if`.

```python
age = int(input("Сколько тебе лет? "))

if age >= 18:
    print("Добро пожаловать!")
else:
    print("Извини, вход с 18 лет")
```

РЕЗУЛЬТАТ

```
Сколько тебе лет? 15
Извини, вход с 18 лет
```

Как это устроено:

- после `if` пишется условие и **двоеточие**;
- команды, которые выполняются при выполнении условия, пишутся **с отступом** (4 пробела или одно нажатие Tab);
- `else:` — «иначе», выполняется, если условие не выполнилось.

Важно. В Python отступы — это часть языка. Именно по отступам Python понимает, какие команды относятся к `if`. Неправильный отступ — ошибка `IndentationError` или неправильная работа программы.

Операторы сравнения

Оператор	Значение
`==`	равно
`!=`	не равно
`>`	больше
`<`	меньше
`>=`	больше или равно
`<=`	меньше или равно

> **Важно.** Сравнение — это `==` (два знака). Один знак `=` — это присваивание. Запись `if x = 5:` — ошибка.

Результат сравнения — значение типа `bool`:

```python
print(5 > 3)
print(5 == 3)
print("кот" == "кот")
```

РЕЗУЛЬТАТ

```
True
False
True
```

Несколько вариантов: elif

`elif` — сокращение от «else if», «иначе если». Условия проверяются по очереди сверху вниз, и выполняется только **первая** подходящая ветка.

```python
score = 78

if score >= 90:
    grade = 5
elif score >= 75:
    grade = 4
elif score >= 50:
    grade = 3
else:
    grade = 2

print(f"Оценка: {grade}")
```

РЕЗУЛЬТАТ

```
Оценка: 4
```

Хотя `78 >= 50` тоже верно, сработала только ветка `>= 75`, потому что она стоит раньше.

Вложенные условия

Внутри `if` можно написать ещё один `if`. Отступ тогда становится больше:

```python
login = "admin"
password = "1234"

if login == "admin":
    if password == "1234":
        print("Вход выполнен")
    else:
        print("Неверный пароль")
else:
    print("Нет такого пользователя")
```

РЕЗУЛЬТАТ

```
Вход выполнен
```

Короткая запись

Если нужно выбрать одно из двух значений, можно записать условие в одну строку:

```python
age = 15
status = "взрослый" if age >= 18 else "подросток"
print(status)
```

РЕЗУЛЬТАТ

```
подросток
```

1.6 Логические операторы и match-case

and, or, not

Иногда нужно проверить сразу несколько условий.

Оператор	Значение	Когда `True`
`and`	«и»	оба условия верны
`or`	«или»	хотя бы одно верно
`not`	«не»	условие неверно

```python
age = 15
has_ticket = True

if age >= 12 and has_ticket:
    print("Проходите в зал")

day = "суббота"
if day == "суббота" or day == "воскресенье":
    print("Выходной!")

is_raining = False
if not is_raining:
    print("Можно гулять")
```

РЕЗУЛЬТАТ

```
Проходите в зал
Выходной!
Можно гулять
```

Проверка, что число в диапазоне, в Python записывается красиво — как в математике:

```python
x = 7
if 1 <= x <= 10:
    print("Число от 1 до 10")
```

РЕЗУЛЬТАТ

```
Число от 1 до 10
```

Пример: проверка високосного года. Год високосный, если делится на 4, но не делится на 100 — или делится на 400.

```python
year = 2024
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print("Високосный")
else:
    print("Обычный")
```

РЕЗУЛЬТАТ

```
Високосный
```

Истинность значений

В условии можно писать не только сравнения. Python считает «ложью» (`False`): `0`, пустую строку `""`, пустой список `[]` и `None`. Всё остальное — «истина».

```python
name = input("Имя: ")
if name:
    print(f"Привет, {name}!")
else:
    print("Ты ничего не ввёл")
```

РЕЗУЛЬТАТ

```
Имя:
Ты ничего не ввёл
```

match-case: выбор из многих вариантов

Когда одну переменную нужно сравнить со многими значениями, цепочка из `elif` становится длинной. Для этого есть `match-case`:

```python
command = "вверх"

match command:
    case "вверх":
        print("Идём на север")
    case "вниз":
        print("Идём на юг")
    case "выход" | "стоп":
        print("Игра окончена")
    case _:
        print("Неизвестная команда")
```

РЕЗУЛЬТАТ

```
Идём на север
```

- `case "выход" | "стоп"` — сработает для любого из вариантов;
- `case _` — «всё остальное», как `else`.

> **Совет.** `match-case` работает в Python 3.10 и новее. Проверь версию командой `python --version`.

1.7 Цикл for и range

Цикл повторяет команды несколько раз. Без циклов, чтобы вывести числа от 1 до 100, пришлось бы написать 100 строк `print`.

Цикл for

```python
for i in range(5):
    print("Привет!", i)
```

РЕЗУЛЬТАТ

```
Привет! 0
Привет! 1
Привет! 2
Привет! 3
Привет! 4
```

`range(5)` выдаёт числа `0, 1, 2, 3, 4` — пять чисел, **начиная с нуля**, и 5 в них не входит. На каждом шаге (**итерации**) очередное число попадает в переменную `i`, и выполняется тело цикла — команды с отступом.

Варианты range

Запись	Числа
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8
`range(10, 0, -1)`	10, 9, 8, ..., 1

Конец диапазона **никогда не входит** в него. Чтобы получить числа от 1 до 10 включительно, пишут `range(1, 11)`.

```python
for i in range(10, 0, -3):
    print(i, end=" ")
print("Пуск!")
```

РЕЗУЛЬТАТ

```
10 7 4 1 Пуск!
```

Накопление результата

Частый приём: завести переменную до цикла и менять её на каждом шаге.

```python
total = 0
for i in range(1, 101):
    total += i
print(f"Сумма чисел от 1 до 100: {total}")
```

РЕЗУЛЬТАТ

```
Сумма чисел от 1 до 100: 5050
```

Перебор строки

`for` умеет перебирать не только числа, но и символы строки:

```python
for letter in "Python":
    print(letter, end="-")
```

РЕЗУЛЬТАТ

```
P-y-t-h-o-n-
```

> **Совет.** Если номер шага не нужен, переменную цикла принято называть `_`: `for _ in range(3): print("Ура!")`.

1.8 Цикл while, break и continue

Цикл while

`while` повторяет команды, **пока условие верно**. Он нужен, когда заранее неизвестно, сколько будет повторений.

```python
password = ""
while password != "python":
    password = input("Пароль: ")
print("Доступ разрешён")
```

РЕЗУЛЬТАТ

```
Пароль: 12345
Пароль: qwerty
Пароль: python
Доступ разрешён
```

Цикл проверяет условие **перед** каждым шагом. Когда пользователь ввёл правильный пароль, условие стало ложным, и программа пошла дальше.

> **Важно.** Если условие никогда не станет ложным, цикл будет крутиться бесконечно и программа «зависнет». Остановить её можно сочетанием клавиш **Ctrl + C** в терминале. Следи, чтобы внутри цикла менялось что-то, от чего зависит условие.

break: выйти из цикла

`break` сразу прекращает цикл. Часто используют бесконечный цикл `while True` и выходят из него через `break`:

```python
while True:
    text = input("Напиши что-нибудь (или 'выход'): ")
    if text == "выход":
        break
    print(f"Ты написал: {text}")
print("Пока!")
```

РЕЗУЛЬТАТ

```
Напиши что-нибудь (или 'выход'): привет
Ты написал: привет
Напиши что-нибудь (или 'выход'): выход
Пока!
```

continue: перейти к следующему шагу

`continue` пропускает остаток тела цикла и сразу начинает следующую итерацию:

```python
for i in range(1, 11):
    if i % 3 == 0:
        continue        # числа, кратные 3, пропускаем
    print(i, end=" ")
```

РЕЗУЛЬТАТ

```
1 2 4 5 7 8 10
```

Игра «Угадай число»

Соберём вместе `while`, `if` и случайные числа. Модуль `random` умеет выбирать случайное число: `random.randint(1, 100)` — от 1 до 100 включительно.

```python
import random

secret = random.randint(1, 100)
attempts = 0

while True:
    guess = int(input("Твоё число: "))
    attempts += 1
    if guess < secret:
        print("Больше!")
    elif guess > secret:
        print("Меньше!")
    else:
        print(f"Угадал! Попыток: {attempts}")
        break
```

РЕЗУЛЬТАТ

```
Твоё число: 50
Меньше!
Твоё число: 25
Больше!
Твоё число: 37
Угадал! Попыток: 3
```

for или while?

- Знаешь, **сколько раз** повторить (или что перебрать) — `for`.
- Повторяешь, **пока что-то не случится** — `while`.

1.9 Вложенные циклы и случайные числа

Цикл внутри цикла

Внутрь одного цикла можно поставить другой. Внутренний цикл будет полностью проходиться на **каждом** шаге внешнего.

PYTHON

```python
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end="\t")
    print()   # переход на новую строку после каждой строки таблицы
```

РЕЗУЛЬТАТ

```
1       2       3
2       4       6
3       6       9
```

`"\t"` — это символ табуляции: он выравнивает значения по столбцам. А `"\n"` — символ перехода на новую строку.

Вложенные циклы удобны, чтобы рисовать фигуры из символов:

PYTHON

```python
for i in range(1, 5):
    print(" " * (4 - i) + "*" * (2 * i - 1))
```

РЕЗУЛЬТАТ

```
   *
  ***
 *****
*******
```

Здесь используется ещё один трюк: строку можно умножить на число, и она повторится нужное количество раз: `"*" * 3` → `"***"`.

Модуль random

```python
import random

print(random.randint(1, 6))                  # целое от 1 до 6 (кубик)
print(random.random())                       # дробное от 0 до 1
print(random.choice(["камень", "ножницы", "бумага"]))  # случайный элемент
```

РЕЗУЛЬТАТ

```
4
0.7364712141640124
ножницы
```

Каждый запуск даёт новые числа. Если нужно, чтобы «случайные» числа повторялись (например, для проверки), в начале программы пишут `random.seed(42)` — с одинаковым числом последовательность будет одинаковой.

1.10 Ошибки и отладка

Ошибки в программах бывают у всех: и у новичков, и у профессионалов. Главный навык — не бояться их, а быстро находить.

Три вида ошибок

1. **Синтаксическая ошибка** — код написан не по правилам языка: забыта скобка, кавычка или двоеточие. Программа даже не запускается.
2. **Ошибка выполнения** — код правильный, но во время работы случилось что-то невозможное: деление на ноль, превращение слова в число. Программа падает на этой строке.
3. **Логическая ошибка** — программа работает и не падает, но результат неправильный. Самые коварные: Python о них не сообщает.

Как читать сообщение об ошибке

```python
numbers = [1, 2, 3]
print(numbers[5])
```

РЕЗУЛЬТАТ

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    print(numbers[5])
          ~~~~~~~^^^
IndexError: list index out of range
```

Читай **снизу вверх**:

- последняя строка — **тип ошибки** (`IndexError`) и её описание («индекс за пределами списка»);

- выше — **файл и номер строки** (`line 2`) и сама строка, где всё сломалось.

Ошибка	Что обычно значит
`SyntaxError`	опечатка в синтаксисе: скобка, кавычка, двоеточие
`IndentationError`	неправильный отступ
`NameError`	переменная не создана или опечатка в имени
`TypeError`	действие с неподходящим типом: сложить строку и число
`ValueError`	неподходящее значение: `int("абв")`
`ZeroDivisionError`	деление на ноль
`IndexError`	обращение к несуществующему элементу списка
`KeyError`	в словаре нет такого ключа

> **Совет.** Если не понимаешь ошибку, скопируй последнюю строку сообщения и найди её в интернете. Почти наверняка кто-то уже сталкивался с тем же.

Отладка через print

Самый простой способ найти логическую ошибку — вывести значения переменных в подозрительных местах:

```python
total = 0
for i in range(1, 5):
    total = i            # ошибка: должно быть total += i
    print(f"шаг {i}: total = {total}")
print(total)
```

```
РЕЗУЛЬТАТ
шаг 1: total = 1
шаг 2: total = 2
шаг 3: total = 3
шаг 4: total = 4
4
```

По выводу сразу видно, что `total` не накапливается, а каждый раз перезаписывается. После исправления отладочные `print` удаляют.

Отладчик

В VS Code есть **отладчик** — инструмент, который позволяет выполнять программу по шагам и смотреть значения всех переменных.

1. Кликни слева от номера строки — появится красная точка, это **точка останова**.
2. Запусти программу через **Run → Start Debugging** (F5).
3. Программа остановится на этой строке. Слева видно все переменные и их значения.
4. Кнопка **Step Over** (F10) выполняет следующую строку.

Ещё один отличный инструмент — сайт **Python Tutor** (pythontutor.com). Вставляешь туда код и смотришь картинкой, как программа выполняется шаг за шагом.

> **Запомни.** Не меняй код наугад. Сначала пойми, *где* и *почему* ошибка: прочитай сообщение, выведи переменные, пройди отладчиком. Потом исправляй.

ЧАСТЬ 2

Строки и коллекции

Одна переменная хранит одно значение. Но в реальных программах данных много: список игроков, оценки за четверть, слова для «Виселицы». Для этого в Python есть коллекции — списки, кортежи, словари и множества. А начнём со строк, потому что они устроены похоже.

2.1 Строки: индексы и срезы

Строка — это последовательность символов. У каждого символа есть **номер** (индекс), и нумерация начинается **с нуля**.

```
 П   р   и   в   е   т
 0   1   2   3   4   5
-6  -5  -4  -3  -2  -1
```

PYTHON

```python
word = "Привет"
print(word[0])     # первый символ
print(word[3])
print(word[-1])    # последний символ
print(len(word))   # длина строки
```

РЕЗУЛЬТАТ

```
П
в
т
6
```

Отрицательные индексы считаются с конца: `-1` — последний символ, `-2` — предпоследний. Это удобно, когда длина строки заранее неизвестна.

> **Важно.** Последний символ строки длиной 6 имеет индекс 5, а не 6. Обращение `word[6]` вызовет ошибку `IndexError`.

Срезы

Срез — это кусок строки. Записывается как `строка[начало:конец]`, и, как в `range`, конец **не входит**.

```python
word = "Программирование"
print(word[0:9])     # с 0 по 8
print(word[:5])      # с начала до 4
print(word[9:])      # с 9 до конца
print(word[-4:])     # последние 4 символа
```

РЕЗУЛЬТАТ

```
Программи
Прогр
рование
ание
```

У среза есть третий параметр — **шаг**: `строка[начало:конец:шаг]`.

```python
word = "Python"
print(word[::2])    # каждый второй символ
print(word[::-1])   # строка задом наперёд
```

РЕЗУЛЬТАТ

```
Pto
nohtyP
```

Запись `[::-1]` — самый короткий способ перевернуть строку. С её помощью легко проверить слово на палиндром (слово, которое читается одинаково в обе стороны):

```python
word = "шалаш"
if word == word[::-1]:
    print("Палиндром!")
```

РЕЗУЛЬТАТ

```
Палиндром!
```

Строки неизменяемы

Поменять один символ в строке нельзя:

```python
word = "кот"
word[0] = "р"
```

РЕЗУЛЬТАТ

```
TypeError: 'str' object does not support item assignment
```

Зато можно собрать новую строку и положить её в ту же переменную:

```python
word = "кот"
word = "р" + word[1:]
print(word)
```

РЕЗУЛЬТАТ

```
рот
```

Операции со строками

```python
print("Пит" + "он")       # склеивание
print("ха" * 3)           # повторение
print("кот" in "котлета") # есть ли подстрока
```

РЕЗУЛЬТАТ

```
Питон
хахаха
True
```

2.2 Методы строк

Метод — это функция, которая «принадлежит» значению. Вызывается через точку: `строка.метод()`. Методы строк не меняют исходную строку, а возвращают новую.

Регистр и пробелы

```python
text = "  Привет, Мир!  "
print(text.upper())     # все заглавные
print(text.lower())     # все строчные
print(text.strip())     # убрать пробелы по краям
print("python".capitalize())  # первая буква заглавная
```

РЕЗУЛЬТАТ

```
  ПРИВЕТ, МИР!
  привет, мир!
Привет, Мир!
Python
```

Совет. Чтобы сравнивать ввод пользователя без учёта регистра и лишних пробелов, пиши `answer.strip().lower() == "да"`. Тогда подойдут и «Да», и « ДА ».

Поиск и замена

```python
text = "Мама мыла раму"
print(text.find("мыла"))      # индекс начала подстроки (-1, если нет)
print(text.count("а"))        # сколько раз встречается
print(text.replace("раму", "окно"))
print(text.startswith("Мама"))
print(text.endswith("!"))
```

РЕЗУЛЬТАТ

```
5
4
Мама мыла окно
True
False
```

Проверка содержимого

```python
print("12345".isdigit())   # только цифры?
print("Python".isalpha())  # только буквы?
print("ABC".isupper())     # все заглавные?
print(" ".isspace())       # только пробелы?
```

РЕЗУЛЬТАТ

```
True
True
True
True
```

split и join: строка ↔ список

`split()` разрезает строку на части и возвращает **список**. Без аргументов режет по пробелам.

```python
sentence = "я люблю писать код"
words = sentence.split()
print(words)
print(len(words), "слова")

date = "12.05.2026"
print(date.split("."))
```

РЕЗУЛЬТАТ

```
['я', 'люблю', 'писать', 'код']
4 слова
['12', '05', '2026']
```

`join()` делает обратное — склеивает список строк, вставляя между ними разделитель:

```python
words = ["Python", "Java", "C#"]
print(", ".join(words))
print(" -> ".join(words))
```

РЕЗУЛЬТАТ

```
Python, Java, C#
Python -> Java -> C#
```

Перебор символов и коды символов

У каждого символа есть числовой код. `ord()` даёт код символа, `chr()` — символ по коду. На этом построен **шифр Цезаря** — каждая буква сдвигается на несколько позиций:

```python
text = "hello"
result = ""
for ch in text:
    result += chr(ord(ch) + 3)
print(result)
```

РЕЗУЛЬТАТ

```
khoor
```

2.3 Списки

Список — это упорядоченный набор значений. Записывается в квадратных скобках через запятую.

```python
fruits = ["яблоко", "банан", "киви"]
scores = [5, 4, 5, 3, 5]
mixed = [1, "два", 3.0, True]   # можно смешивать типы
empty = []                      # пустой список
```

Доступ к элементам

Индексы и срезы работают так же, как у строк:

```python
fruits = ["яблоко", "банан", "киви", "манго"]
print(fruits[0])
print(fruits[-1])
print(fruits[1:3])
print(len(fruits))
```

РЕЗУЛЬТАТ

```
яблоко
манго
['банан', 'киви']
4
```

Списки можно менять

В отличие от строк, элементы списка можно заменять, добавлять и удалять.

```python
fruits = ["яблоко", "банан", "киви"]
fruits[1] = "груша"          # заменить
fruits.append("манго")       # добавить в конец
fruits.insert(0, "лимон")    # вставить на позицию 0
print(fruits)

fruits.remove("киви")        # удалить по значению
last = fruits.pop()          # удалить последний и вернуть его
print(fruits)
print("Удалили:", last)
```

РЕЗУЛЬТАТ

```
['лимон', 'яблоко', 'груша', 'киви', 'манго']
['лимон', 'яблоко', 'груша']
Удалили: манго
```

Метод	Что делает
`append(x)`	добавить `x` в конец
`insert(i, x)`	вставить `x` на позицию `i`
`remove(x)`	удалить первый элемент со значением `x`
`pop()` / `pop(i)`	удалить последний (или `i`-й) и вернуть его
`index(x)`	индекс первого элемента со значением `x`
`count(x)`	сколько раз встречается `x`
`sort()`	отсортировать список
`reverse()`	перевернуть список
`clear()`	удалить все элементы

Сортировка

```python
scores = [3, 5, 1, 4, 2]
scores.sort()                  # сортирует сам список
print(scores)
scores.sort(reverse=True)      # по убыванию
print(scores)

names = ["Петя", "Аня", "Вика"]
print(sorted(names))           # возвращает НОВЫЙ отсортированный список
print(names)                   # исходный не изменился
```

РЕЗУЛЬТАТ

```
[1, 2, 3, 4, 5]
[5, 4, 3, 2, 1]
['Аня', 'Вика', 'Петя']
['Петя', 'Аня', 'Вика']
```

Перебор списка

```python
fruits = ["яблоко", "банан", "киви"]

for fruit in fruits:
    print(fruit)

for i, fruit in enumerate(fruits, start=1):   # с номером
    print(f"{i}. {fruit}")
```

РЕЗУЛЬТАТ

```
яблоко
банан
киви
1. яблоко
2. банан
3. киви
```

Проверка и полезные функции

```python
scores = [5, 4, 5, 3, 5]
print(4 in scores)          # есть ли элемент
print(sum(scores))          # сумма
print(max(scores), min(scores))
print(sum(scores) / len(scores))   # среднее
```

РЕЗУЛЬТАТ

```
True
22
5 3
4.4
```

Ловушка копирования

```python
a = [1, 2, 3]
b = a          # b — это НЕ копия, а второе имя того же списка
b.append(4)
print(a)
```

РЕЗУЛЬТАТ

```
[1, 2, 3, 4]
```

Переменные `a` и `b` указывают на **один и тот же** список. Чтобы получить независимую копию, используй `a.copy()` или `a[:]`:

```python
a = [1, 2, 3]
b = a.copy()
b.append(4)
print(a, b)
```

РЕЗУЛЬТАТ

```
[1, 2, 3] [1, 2, 3, 4]
```

> **Запомни.** Список — изменяемый. `b = a` не копирует список, а даёт ему второе имя.

2.4 Обработка списков

`sum`, `max` и `min` — готовые функции. Но важно понимать, как такие вещи делаются вручную: на этом построены почти все алгоритмы.

Поиск максимума и его позиции

```python
temps = [12, 18, 9, 21, 15]

best = temps[0]
best_day = 0
for i in range(len(temps)):
    if temps[i] > best:
        best = temps[i]
        best_day = i

print(f"Самый тёплый день: {best_day + 1}, температура {best}")
```

РЕЗУЛЬТАТ

```
Самый тёплый день: 4, температура 21
```

Идея: запоминаем «лучшее пока что» и сравниваем с каждым элементом.

Подсчёт и фильтрация

```python
scores = [5, 3, 4, 5, 2, 5, 4]

fives = 0
for s in scores:
    if s == 5:
        fives += 1
print("Пятёрок:", fives)

good = []
for s in scores:
    if s >= 4:
        good.append(s)
print("Хорошие оценки:", good)
```

РЕЗУЛЬТАТ

```
Пятёрок: 3
Хорошие оценки: [5, 4, 5, 5, 4]
```

Генераторы списков

То же самое, что в последнем примере, можно записать в одну строку — **генератором списка** (list comprehension):

```python
scores = [5, 3, 4, 5, 2, 5, 4]
good = [s for s in scores if s >= 4]
print(good)

squares = [x ** 2 for x in range(1, 6)]
print(squares)

words = ["кот", "собака", "ёж"]
lengths = [len(w) for w in words]
print(lengths)
```

РЕЗУЛЬТАТ

```
[5, 4, 5, 5, 4]
[1, 4, 9, 16, 25]
[3, 6, 2]
```

Читается так: «`s` для каждого `s` из `scores`, если `s >= 4`».

Перебор двух списков сразу: zip

```python
names = ["Аня", "Петя", "Вика"]
scores = [92, 78, 85]

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

РЕЗУЛЬТАТ

```
Аня: 92
Петя: 78
Вика: 85
```

2.5 Двумерные списки

Элементами списка могут быть другие списки. Так получается **таблица** или игровое поле.

```python
board = [
    ["X", "O", "X"],
    [" ", "X", "O"],
    ["O", " ", "X"],
]

print(board[0])      # первая строка
print(board[1][2])   # строка 1, столбец 2
```

РЕЗУЛЬТАТ

```
['X', 'O', 'X']
O
```

Первый индекс — номер строки, второй — номер столбца.

Вывод поля

```python
board = [
    ["X", "O", "X"],
    [" ", "X", "O"],
    ["O", " ", "X"],
]

for row in board:
    print(" | ".join(row))
```

РЕЗУЛЬТАТ

```
X | O | X
  | X | O
O |   | X
```

Как правильно создать пустое поле

```python
size = 3
board = [[" "] * size for _ in range(size)]
board[0][0] = "X"
print(board)
```

РЕЗУЛЬТАТ

```
'X', ' ', ' '], [' ', ' ', ' '], [' ', ' ', ' '
```

Важно. Не создавай поле так: `[[" "] * 3] * 3`. Получится три ссылки на **одну и ту же** строку, и при изменении одной клетки изменится весь столбец.

```python
board = [[" "] * 3] * 3    # так делать НЕЛЬЗЯ
board[0][0] = "X"
print(board)
```

РЕЗУЛЬТАТ

```
'X', ' ', ' '], ['X', ' ', ' '], ['X', ' ', ' '
```

Перебор всех клеток

```python
grades = [
    [5, 4, 5],    # оценки Ани
    [3, 4, 4],    # оценки Пети
]

for i in range(len(grades)):
    for j in range(len(grades[i])):
        print(f"ученик {i}, предмет {j}: {grades[i][j]}")
```

РЕЗУЛЬТАТ

```
ученик 0, предмет 0: 5
ученик 0, предмет 1: 4
ученик 0, предмет 2: 5
ученик 1, предмет 0: 3
ученик 1, предмет 1: 4
ученик 1, предмет 2: 4
```

2.6 Кортежи

Кортеж (`tuple`) похож на список, но его **нельзя изменить** после создания. Записывается в круглых скобках.

```python
point = (10, 20)
colors = ("красный", "зелёный", "синий")
print(point[0])
print(len(colors))
```

РЕЗУЛЬТАТ

```
10
3
```

Зачем нужен кортеж, если есть список? Когда данные не должны меняться: координаты, цвет в формате RGB `(255, 0, 0)`, дата. Кортеж защищает от случайных изменений.

Распаковка

Значения кортежа можно разложить по переменным:

```python
point = (10, 20)
x, y = point
print(x, y)
```

РЕЗУЛЬТАТ

```
10 20
```

Распаковка работает с любыми коллекциями нужной длины, а запись `x, y = y, x` из темы 1.1 — это как раз создание кортежа и его распаковка.

2.7 Словари

Словарь (`dict`) хранит пары **ключ → значение**. По ключу можно быстро найти значение — как в настоящем словаре по слову находят перевод.

```python
player = {
    "name": "Rex",
    "level": 12,
    "hp": 100,
}
print(player["name"])
print(player["level"])
```

РЕЗУЛЬТАТ

```
Rex
12
```

Ключи обычно строки, но могут быть и числами или кортежами. Значения — что угодно.

Изменение словаря

```python
player = {"name": "Rex", "level": 12}
player["level"] = 13         # изменить значение
player["gold"] = 50          # добавить новую пару
del player["name"]           # удалить пару
print(player)
```

РЕЗУЛЬТАТ

```
{'level': 13, 'gold': 50}
```

Если ключа нет

```python
player = {"name": "Rex"}
print(player["gold"])
```

РЕЗУЛЬТАТ

```
KeyError: 'gold'
```

Чтобы не получить ошибку, есть два способа: проверить ключ через `in` или использовать метод `get()`, который вернёт значение по умолчанию:

```python
player = {"name": "Rex"}
if "gold" in player:
    print(player["gold"])
print(player.get("gold", 0))   # если ключа нет, вернёт 0
```

РЕЗУЛЬТАТ

```
0
```

Перебор словаря

```python
prices = {"хлеб": 50, "молоко": 90, "сыр": 300}

for item in prices:                # перебор ключей
    print(item)

for item, price in prices.items():  # ключи и значения
    print(f"{item}: {price} ₽")

print(list(prices.keys()))
print(list(prices.values()))
```

РЕЗУЛЬТАТ

```
хлеб
молоко
сыр
хлеб: 50 ₽
молоко: 90 ₽
сыр: 300 ₽
['хлеб', 'молоко', 'сыр']
[50, 90, 300]
```

Пример: частотный словарь

Посчитаем, сколько раз встречается каждое слово:

```python
text = "кот пёс кот кот рыба пёс"
counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

РЕЗУЛЬТАТ

```
{'кот': 3, 'пёс': 2, 'рыба': 1}
```

Список словарей

Самая частая структура данных в реальных программах — список, в котором каждый элемент — словарь. Так хранят товары в магазине, игроков, книги.

```python
movies = [
    {"title": "Интерстеллар", "year": 2014, "rating": 8.6},
    {"title": "Начало", "year": 2010, "rating": 8.7},
    {"title": "Дюна", "year": 2021, "rating": 7.9},
]

for movie in movies:
    if movie["rating"] > 8:
        print(f"{movie['title']} ({movie['year']})")
```

РЕЗУЛЬТАТ

```
Интерстеллар (2014)
Начало (2010)
```

Важно. Внутри f-строки в двойных кавычках ключи словаря пиши в одинарных: `f"{movie['title']}"`.

2.8 Множества

Множество (`set`) — набор **уникальных** элементов без порядка. Повторы в множестве автоматически убираются.

```python
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)

names = ["Аня", "Петя", "Аня", "Вика", "Петя"]
unique = set(names)
print(len(unique), "уникальных имени")
```

РЕЗУЛЬТАТ

```
{1, 2, 3}
3 уникальных имени
```

Важно. Пустое множество создаётся как `set()`, а не `{}` — пустые фигурные скобки создают пустой **словарь**.

Добавление и проверка

```python
visited = set()
visited.add("Москва")
visited.add("Казань")
visited.add("Москва")       # уже есть, ничего не изменится
print(len(visited))
print("Казань" in visited)
```

РЕЗУЛЬТАТ

```
2
True
```

Проверка `in` в множестве работает **очень быстро** — намного быстрее, чем в списке. Это важно, когда данных много.

Операции над множествами

```python
anya = {"Петя", "Вика", "Миша", "Лёша"}
petya = {"Вика", "Миша", "Дима"}

print(anya & petya)   # общие друзья (пересечение)
print(anya | petya)   # все друзья вместе (объединение)
print(anya - petya)   # друзья Ани, которых не знает Петя (разность)
```

РЕЗУЛЬТАТ

```
{'Вика', 'Миша'}
{'Петя', 'Вика', 'Миша', 'Лёша', 'Дима'}
{'Петя', 'Лёша'}
```

У множества нет порядка, поэтому элементы могут выводиться в другом порядке, чем ты их добавлял.

2.9 Какую коллекцию выбрать

Коллекция	Запись	Порядок	Можно менять	Повторы	Когда использовать
Список	`[1, 2, 3]`	да	да	да	набор однотипных элементов: оценки, игроки, ходы
Кортеж	`(1, 2)`	да	нет	да	данные, которые не должны меняться: координаты, цвет
Словарь	`{"a": 1}`	да	да	ключи — нет	поиск по имени: настройки, свойства объекта
Множество	`{1, 2}`	нет	да	нет	уникальные значения и быстрая проверка «есть ли»

> **Запомни.** Нужен порядок — список. Нужен поиск по ключу — словарь. Нужна уникальность — множество. Нужно защитить от изменений — кортеж.

ЧАСТЬ 3

Функции и модули

Когда программа вырастает до сотен строк, в ней становится трудно разобраться. Функции позволяют разбить её на небольшие понятные части, а модули — разложить эти части по разным файлам.

3.1 Функции

Функция — это именованный кусок кода, который можно вызывать много раз. Ты уже пользовался готовыми функциями: `print()`, `len()`, `input()`. Теперь научимся писать свои.

PYTHON

```python
def greet():
    print("Привет!")
    print("Рад тебя видеть")

greet()
greet()
```

РЕЗУЛЬТАТ

```
Привет!
Рад тебя видеть
Привет!
Рад тебя видеть
```

- `def` — слово, с которого начинается **объявление** функции;
- `greet` — имя функции (правила те же, что для переменных);
- тело функции пишется с отступом;
- `greet()` — **вызов** функции. Пока функцию не вызвали, её код не выполняется.

> **Важно.** Функцию нужно объявить **выше** того места, где её вызывают. Иначе будет ошибка `NameError`.

Параметры

В функцию можно передавать данные. Переменные в скобках при объявлении называются **параметрами**, а значения при вызове — **аргументами**.

```python
def greet(name):
    print(f"Привет, {name}!")

greet("Аня")
greet("Петя")
```

РЕЗУЛЬТАТ

```
Привет, Аня!
Привет, Петя!
```

Параметров может быть несколько:

```python
def show_score(name, score):
    print(f"{name}: {score} очков")

show_score("Аня", 120)
```

РЕЗУЛЬТАТ

```
Аня: 120 очков
```

return: функция возвращает результат

Часто функция должна не напечатать результат, а **вернуть** его, чтобы им можно было дальше пользоваться. Для этого есть `return`.

```python
def square(x):
    return x * x

result = square(5)
print(result)
print(square(3) + square(4))
```

РЕЗУЛЬТАТ

```
25
25
```

Когда выполняется `return`, функция **сразу заканчивается**. Код после `return` не выполнится.

```python
def check_age(age):
    if age < 0:
        return "Ошибка: возраст не бывает отрицательным"
    if age >= 18:
        return "Взрослый"
    return "Ребёнок"

print(check_age(25))
print(check_age(-3))
```

РЕЗУЛЬТАТ

```
Взрослый
Ошибка: возраст не бывает отрицательным
```

print или return?

Это частая путаница:

```python
def add_print(a, b):
    print(a + b)

def add_return(a, b):
    return a + b

x = add_print(2, 3)
y = add_return(2, 3)
print("x =", x)
print("y =", y)
```

РЕЗУЛЬТАТ

```
5
x = None
y = 5
```

Функция без `return` ничего не возвращает — точнее, возвращает `None`. Поэтому в `x` оказалось `None`, хотя на экране и появилось 5.

> **Запомни.** `print` — показать человеку. `return` — отдать результат программе для дальнейшей работы.

Несколько значений

Функция может вернуть сразу несколько значений через запятую (на самом деле это кортеж):

```python
def min_max(numbers):
    return min(numbers), max(numbers)

low, high = min_max([4, 9, 1, 7])
print(low, high)
```

РЕЗУЛЬТАТ

```
1 9
```

Описание функции

Первой строкой в функции можно написать **строку документации** — что делает функция. Редактор покажет её, когда наведёшь мышь на вызов функции.

```python
def average(numbers):
    """Возвращает среднее арифметическое списка чисел."""
    return sum(numbers) / len(numbers)
```

Зачем вообще нужны функции

- **Не повторяться.** Код, который нужен в нескольких местах, пишется один раз.
- **Понятность.** `draw_board(board)` читается легче, чем 10 строк циклов.
- **Проще искать ошибки.** Каждую функцию можно проверить отдельно.

> **Совет.** Правило «одна функция — одна задача». Если функцию трудно назвать коротко или она не помещается на экран — раздели её на несколько.

3.2 Параметры функций подробнее

Значения по умолчанию

Параметру можно задать значение по умолчанию. Тогда при вызове его можно не указывать.

```python
def power(base, exp=2):
    return base ** exp

print(power(5))      # exp = 2 по умолчанию
print(power(2, 10))
```

РЕЗУЛЬТАТ

```
25
1024
```

Именованные аргументы

Аргументы можно передавать по имени параметра. Тогда порядок не важен, и код читается понятнее:

```python
def create_player(name, hp=100, speed=5):
    print(f"{name}: здоровье {hp}, скорость {speed}")

create_player("Rex")
create_player("Bolt", speed=10)
create_player(hp=50, name="Tank")
```

РЕЗУЛЬТАТ

```
Rex: здоровье 100, скорость 5
Bolt: здоровье 100, скорость 10
Tank: здоровье 50, скорость 5
```

Так уже работает `print()`: `sep` и `end` — это именованные аргументы со значениями по умолчанию.

Любое количество аргументов: *args

Если перед параметром поставить `*`, функция примет сколько угодно аргументов, а в параметр попадёт кортеж:

```python
def total(*numbers):
    result = 0
    for n in numbers:
        result += n
    return result

print(total(1, 2))
print(total(5, 10, 15, 20))
```

РЕЗУЛЬТАТ

```
3
50
```

С двумя звёздочками `**kwargs` функция принимает любые именованные аргументы и получает их в виде словаря. Встречается реже, но знать полезно.

3.3 Область видимости

Локальные и глобальные переменные

Переменные, созданные **внутри** функции, — **локальные**: они существуют только во время работы функции и снаружи не видны.

```python
def test():
    secret = 42
    print(secret)

test()
print(secret)
```

РЕЗУЛЬТАТ

```
42
NameError: name 'secret' is not defined
```

Переменные, созданные вне функций, — **глобальные**. Функция может их читать, но если попытается присвоить им новое значение, Python создаст новую локальную переменную с тем же именем:

```python
score = 0

def add_points():
    score = 10          # это новая ЛОКАЛЬНАЯ переменная
    print("внутри:", score)

add_points()
print("снаружи:", score)
```

РЕЗУЛЬТАТ

```
внутри: 10
снаружи: 0
```

Изменить глобальную переменную можно с помощью слова `global`, но это **плохая привычка**: в большой программе становится непонятно, кто и где меняет данные. Правильнее передать значение в функцию и вернуть результат:

```python
def add_points(score, points):
    return score + points

score = 0
score = add_points(score, 10)
print(score)
```

РЕЗУЛЬТАТ

```
10
```

Функция может изменить список

С изменяемыми объектами (списками, словарями) есть особенность: функция получает **тот же самый** список, а не копию. Поэтому изменения видны снаружи.

```python
def add_bonus(scores):
    scores.append(100)

my_scores = [50, 70]
add_bonus(my_scores)
print(my_scores)
```

РЕЗУЛЬТАТ

```
[50, 70, 100]
```

Иногда это удобно, иногда — источник ошибок. Если исходный список трогать нельзя, работай с копией: `scores = scores.copy()`.

> **Важно.** Никогда не используй изменяемое значение по умолчанию: `def add(item, items=[])`. Этот список создаётся один раз и «запоминает» все прошлые вызовы. Правильно так: `def add(item, items=None)`, а внутри — `if items is None: items = []`.

3.4 Обработка ошибок: try и except

Когда во время работы программы случается ошибка (например, пользователь ввёл буквы вместо числа), Python создаёт **исключение**, и программа падает. Но исключение можно **перехватить** и обработать.

```python
try:
    age = int(input("Возраст: "))
    print(f"Через год будет {age + 1}")
except ValueError:
    print("Нужно ввести число!")
```

РЕЗУЛЬТАТ

```
Возраст: много
Нужно ввести число!
```

- в блок `try` помещают код, который может сломаться;
- если внутри случилась ошибка указанного типа, Python не падает, а переходит в блок `except`;
- если ошибки не было, `except` пропускается.

Ввод, который не сломать

Объединим `try` и цикл. Такая функция будет спрашивать, пока не получит правильное число:

```python
def ask_int(prompt, min_value, max_value):
    while True:
        try:
            value = int(input(prompt))
        except ValueError:
            print("Это не число, попробуй ещё раз")
            continue
        if min_value <= value <= max_value:
            return value
        print(f"Нужно число от {min_value} до {max_value}")

choice = ask_int("Выбери пункт меню (1-3): ", 1, 3)
print("Ты выбрал", choice)
```

РЕЗУЛЬТАТ

```
Выбери пункт меню (1-3): два
Это не число, попробуй ещё раз
Выбери пункт меню (1-3): 7
Нужно число от 1 до 3
Выбери пункт меню (1-3): 2
Ты выбрал 2
```

> **Совет.** Сохрани эту функцию себе: она пригодится почти в каждой консольной программе.

Несколько видов ошибок

```python
try:
    a = int(input("Делимое: "))
    b = int(input("Делитель: "))
    print(a / b)
except ValueError:
    print("Нужно ввести числа")
except ZeroDivisionError:
    print("На ноль делить нельзя")
```

РЕЗУЛЬТАТ

```
Делимое: 10
Делитель: 0
На ноль делить нельзя
```

else и finally

- `else` — выполняется, если ошибки **не было**;
- `finally` — выполняется **всегда**, была ошибка или нет.

```python
try:
    number = int("42")
except ValueError:
    print("Ошибка")
else:
    print("Получилось:", number)
finally:
    print("Проверка окончена")
```

РЕЗУЛЬТАТ

```
Получилось: 42
Проверка окончена
```

Свои ошибки: raise

Свою ошибку можно создать командой `raise`. Это нужно, когда функция получила неправильные данные и не может продолжать работу:

```python
def set_volume(level):
    if not 0 <= level <= 100:
        raise ValueError("Громкость должна быть от 0 до 100")
    print(f"Громкость: {level}")

try:
    set_volume(150)
except ValueError as error:
    print("Ошибка:", error)
```

РЕЗУЛЬТАТ

```
Ошибка: Громкость должна быть от 0 до 100
```

`as error` позволяет получить текст ошибки.

> **Важно.** Не пиши пустой `except:` без типа ошибки. Он перехватит вообще всё, даже опечатки в коде, и найти настоящую проблему станет очень трудно.

3.5 Стандартная библиотека

Вместе с Python устанавливается огромная **стандартная библиотека** — набор готовых **модулей** для самых разных задач. Модуль подключается командой `import`.

Способы подключения

```python
import math                 # подключить модуль целиком
print(math.sqrt(25))

from random import randint  # подключить только одну функцию
print(randint(1, 6))

import datetime as dt       # подключить под коротким именем
print(dt.date.today())
```

```
РЕЗУЛЬТАТ
5.0
3
2026-10-09
```

Самые полезные модули

Модуль	Для чего	Пример
`math`	математика	`math.sqrt(2)`, `math.pi`
`random`	случайные числа	`random.randint(1, 6)`, `random.choice(список)`, `random.shuffle(список)`
`time`	время и паузы	`time.sleep(1)` — пауза 1 секунда
`datetime`	даты	`datetime.date.today()`
`string`	наборы символов	`string.ascii_letters`, `string.digits`
`os`	работа с системой	`os.listdir()` — файлы в папке
`json`, `csv`	форматы данных	см. часть 5

Пример: генератор паролей

```python
import random
import string

def make_password(length=12):
    symbols = string.ascii_letters + string.digits + "!@#$%"
    return "".join(random.choice(symbols) for _ in range(length))

print(make_password())
print(make_password(6))
```

РЕЗУЛЬТАТ

```
k7R!pQ2x@Lm9
a8F#d1
```

Пример: таймер с обратным отсчётом

```python
import time

for i in range(3, 0, -1):
    print(i)
    time.sleep(1)       # ждём одну секунду
print("Старт!")
```

РЕЗУЛЬТАТ

```
3
2
1
Старт!
```

> **Совет.** Все модули стандартной библиотеки описаны на сайте docs.python.org. Там есть раздел на русском языке. Учись искать ответы в документации — это главный навык программиста.

3.6 Свои модули

Любой файл `.py` — это модуль. Если в папке проекта лежит файл `utils.py`, его можно подключить из другого файла.

```python
# файл utils.py
def ask_int(prompt, min_value, max_value):
    while True:
        try:
            value = int(input(prompt))
        except ValueError:
            print("Это не число")
            continue
        if min_value <= value <= max_value:
            return value
        print(f"Нужно от {min_value} до {max_value}")

def line(width=30):
    print("-" * width)
```

```python
# файл main.py
from utils import ask_int, line

line()
age = ask_int("Возраст: ", 1, 120)
line()
```

Так большую программу раскладывают по файлам: в одном — меню, в другом — игра, в третьем — полезные функции.

if name == "main"

Когда модуль импортируется, весь его код **выполняется**. Если в `utils.py` есть проверочный код, он запустится при каждом `import`. Чтобы этого не было, проверочный код прячут в специальное условие:

```python
# файл utils.py
def line(width=30):
    print("-" * width)

if __name__ == "__main__":
    # этот код выполнится, только если запустить сам utils.py
    line()
    print("Проверка модуля")
```

Переменная `__name__` равна `"__main__"`, только когда файл запущен напрямую. При импорте условие ложно, и проверочный код не выполняется.

3.7 pip и виртуальные окружения

Кроме стандартной библиотеки, существуют сотни тысяч модулей, написанных другими людьми. Они лежат на сайте **PyPI** (pypi.org), а устанавливаются программой **pip**.

ТЕРМИНАЛ

```
pip install rich
```

После установки библиотеку можно импортировать. Например, `rich` делает вывод в консоли цветным и красивым:

PYTHON

```python
from rich import print
from rich.table import Table

print("[bold green]Победа![/bold green] Ты набрал [yellow]120[/yellow] очков")

table = Table(title="Рекорды")
table.add_column("Игрок")
table.add_column("Очки")
table.add_row("Аня", "120")
table.add_row("Петя", "95")
print(table)
```

Виртуальное окружение

Если ставить все библиотеки «в общую кучу», разные проекты могут начать мешать друг другу. Поэтому для каждого проекта создают **виртуальное окружение** — отдельную папку со своими библиотеками.

ТЕРМИНАЛ

```
python -m venv .venv             # создать окружение (один раз)
.venv\Scripts\activate           # включить на Windows
source .venv/bin/activate        # включить на macOS и Linux
pip install rich pygame-ce       # библиотеки ставятся только в это окружение
```

Когда окружение включено, в начале строки терминала появляется `(.venv)`. VS Code обычно сам предлагает использовать созданное окружение.

requirements.txt

Чтобы другой человек мог запустить твой проект, список нужных библиотек сохраняют в файл:

ТЕРМИНАЛ

```
pip freeze > requirements.txt    # записать список библиотек
pip install -r requirements.txt  # установить всё из списка
```

3.8 Функции как значения и lambda ДОП. ТЕМА

Функция в Python — тоже значение. Её можно положить в переменную, в список или словарь и передать в другую функцию.

```python
def add(a, b):
    return a + b

def mul(a, b):
    return a * b

operations = {"+": add, "*": mul}
print(operations["+"](2, 3))
print(operations["*"](2, 3))
```

РЕЗУЛЬТАТ

```
5
6
```

Так удобно делать калькулятор или меню: вместо длинной цепочки `if` — словарь «команда → функция».

lambda

`lambda` — короткая функция в одну строку, без имени:

```python
square = lambda x: x * x
print(square(7))
```

РЕЗУЛЬТАТ

```
49
```

Чаще всего `lambda` используют, чтобы указать, **по чему** сортировать:

```python
players = [
    {"name": "Аня", "score": 120},
    {"name": "Петя", "score": 95},
    {"name": "Вика", "score": 150},
]

best = sorted(players, key=lambda p: p["score"], reverse=True)
for p in best:
    print(p["name"], p["score"])
```

РЕЗУЛЬТАТ

```
Вика 150
Аня 120
Петя 95
```

Параметр `key` получает функцию, которая для каждого элемента говорит, по какому значению его сравнивать. То же самое работает с `max` и `min`:

```python
words = ["кот", "крокодил", "ёж"]
print(max(words, key=len))
```

РЕЗУЛЬТАТ

```
крокодил
```

ЧАСТЬ 4

Алгоритмы

Алгоритм — это точный рецепт решения задачи: последовательность шагов, которая из входных данных получает ответ. В этой части разберём классические алгоритмы поиска и сортировки и научимся понимать, какой алгоритм быстрее.

4.1 Сложность алгоритмов

Две программы могут решать одну задачу, но одна справится за секунду, а другая — за час. Разница не в компьютере, а в **алгоритме**.

Как измерить время

```python
import time

start = time.perf_counter()
total = sum(range(10_000_000))
end = time.perf_counter()

print(f"Время: {end - start:.3f} с")
```

РЕЗУЛЬТАТ

```
Время: 0.172 с
```

`time.perf_counter()` возвращает точное время в секундах. Разница между двумя замерами — время работы кода. (Подчёркивания в числе `10_000_000` нужны только для удобства чтения.)

Считаем шаги, а не секунды

Время зависит от компьютера, поэтому алгоритмы сравнивают по тому, **как растёт число шагов** при росте объёма данных `n`. Это записывают как **«О-большое»**:

Запись	Название	Как растёт	Пример
O(1)	константная	не зависит от `n`	взять элемент по индексу `a[5]`
O(log n)	логарифмическая	очень медленно	бинарный поиск
O(n)	линейная	вместе с `n`	найти максимум в списке
O(n log n)	—	чуть быстрее, чем `n`	хорошие сортировки, `sorted()`
O(n²)	квадратичная	очень быстро	вложенный цикл по всем парам, пузырёк

Чтобы почувствовать разницу: если `n = 1 000 000`, то алгоритм O(log n) сделает около 20 шагов, O(n) — миллион, а O(n²) — триллион. Последний не закончится и за час.

Список или множество

Проверка `x in список` перебирает элементы по одному — это O(n). А `x in множество` работает за O(1) — мгновенно, как бы велико ни было множество.

```python
import time

numbers_list = list(range(1_000_000))
numbers_set = set(numbers_list)

start = time.perf_counter()
for _ in range(1000):
    999_999 in numbers_list
print(f"Список:    {time.perf_counter() - start:.4f} с")

start = time.perf_counter()
for _ in range(1000):
    999_999 in numbers_set
print(f"Множество: {time.perf_counter() - start:.4f} с")
```

РЕЗУЛЬТАТ

```
Список:    6.2190 с
Множество: 0.0001 с
```

> **Запомни.** Один цикл по данным — O(n). Цикл в цикле по тем же данным — O(n²), и это медленно на больших данных.

4.2 Поиск

Линейный поиск

Самый простой способ найти элемент — проверять все по очереди.

```python
def linear_search(items, target):
    for i in range(len(items)):
        if items[i] == target:
            return i          # нашли — возвращаем индекс
    return -1                 # не нашли

numbers = [7, 3, 9, 1, 5]
print(linear_search(numbers, 9))
print(linear_search(numbers, 4))
```

РЕЗУЛЬТАТ

```
2
-1
```

Сложность O(n): в худшем случае придётся проверить все элементы. Зато работает для любого списка, даже неотсортированного.

Бинарный поиск

Если список **отсортирован**, искать можно намного быстрее. Вспомни игру «Угадай число от 1 до 100». Лучшая стратегия — называть середину: «50?» — «Меньше» — «25?» — «Больше» — «37?»... Каждый вопрос отбрасывает половину вариантов.

```python
def binary_search(items, target):
    left = 0
    right = len(items) - 1
    while left <= right:
        middle = (left + right) // 2
        if items[middle] == target:
            return middle
        if items[middle] < target:
            left = middle + 1       # ищем в правой половине
        else:
            right = middle - 1      # ищем в левой половине
    return -1

numbers = [1, 3, 5, 7, 9, 11, 13, 15]
print(binary_search(numbers, 11))
print(binary_search(numbers, 4))
```

РЕЗУЛЬТАТ

```
5
-1
```

Как это работает на поиске числа 11:

```
[1, 3, 5, 7, 9, 11, 13, 15]   середина 7  < 11 → берём правую половину
             [9, 11, 13, 15]   середина 11 = 11 → нашли!
```

Сложность O(log n): для списка из миллиона элементов — около 20 шагов вместо миллиона.

> **Важно.** Бинарный поиск работает **только на отсортированном** списке. На неотсортированном он вернёт неправильный ответ.

4.3 Простые сортировки

Сортировка — расставить элементы по порядку. В Python есть готовые `sort()` и `sorted()`, но разобраться, как сортировки устроены внутри, — классика программирования и отличная тренировка мышления.

Сортировка пузырьком

Идея: проходим по списку и сравниваем **соседей**. Если левый больше правого — меняем местами. После первого прохода самый большой элемент «всплывает» в конец, как пузырёк. Повторяем, пока список не станет отсортированным.

```python
def bubble_sort(items):
    n = len(items)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
    return items

print(bubble_sort([5, 2, 9, 1, 7]))
```

РЕЗУЛЬТАТ

```
[1, 2, 5, 7, 9]
```

Как меняется список на каждом проходе:

```
[5, 2, 9, 1, 7]   исходный
[2, 5, 1, 7, 9]   после 1-го прохода: 9 на месте
[2, 1, 5, 7, 9]   после 2-го: 7 на месте
[1, 2, 5, 7, 9]   после 3-го: готово
```

Сортировка выбором

Идея: находим **наименьший** элемент и ставим его на первое место. Потом ищем наименьший среди оставшихся и ставим на второе место, и так далее.

```python
def selection_sort(items):
    n = len(items)
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if items[j] < items[min_index]:
                min_index = j
        items[i], items[min_index] = items[min_index], items[i]
    return items

print(selection_sort([5, 2, 9, 1, 7]))
```

РЕЗУЛЬТАТ

```
[1, 2, 5, 7, 9]
```

Сортировка вставками

Идея, как при сортировке карт в руке: берём очередной элемент и **вставляем** его на правильное место среди уже отсортированных слева.

```python
def insertion_sort(items):
    for i in range(1, len(items)):
        current = items[i]
        j = i - 1
        while j >= 0 and items[j] > current:
            items[j + 1] = items[j]     # сдвигаем больший элемент вправо
            j -= 1
        items[j + 1] = current
    return items

print(insertion_sort([5, 2, 9, 1, 7]))
```

PYTHON

РЕЗУЛЬТАТ

```
[1, 2, 5, 7, 9]
```

У всех трёх простых сортировок сложность O(n²): цикл внутри цикла. На 100 элементах это незаметно, на 100 000 — очень долго.

> **Совет.** Чтобы понять сортировку, разыграй её на карточках с числами, прежде чем писать код. А на сайте visualgo.net можно посмотреть анимацию каждой сортировки.

4.4 Рекурсия

Рекурсия — это когда функция вызывает **саму себя**. Звучит странно, но многие задачи так решаются проще всего.

Факториал

Факториал числа `n` (пишется `n!`) — это произведение всех чисел от 1 до `n`: `5! = 5 · 4 · 3 · 2 · 1 = 120`. Заметь, что `5! = 5 · 4!`, а `4! = 4 · 3!` — задача сводится к такой же, но поменьше.

```python
def factorial(n):
    if n == 1:                  # базовый случай: дальше не идём
        return 1
    return n * factorial(n - 1)   # шаг рекурсии

print(factorial(5))
```

PYTHON

РЕЗУЛЬТАТ

```
120
```

Любая рекурсивная функция состоит из двух частей:

1. **Базовый случай** — простейшая ситуация, где ответ известен сразу (`factorial(1) = 1`).
2. **Шаг рекурсии** — сведение задачи к такой же, но меньшей (`factorial(n - 1)`).

Как выполняется `factorial(4)`:

```
factorial(4) = 4 * factorial(3)
                   factorial(3) = 3 * factorial(2)
                                      factorial(2) = 2 * factorial(1)
                                                         factorial(1) = 1
                                      factorial(2) = 2 * 1 = 2
                   factorial(3) = 3 * 2 = 6
factorial(4) = 4 * 6 = 24
```

> **Важно.** Без базового случая функция будет вызывать себя бесконечно, пока Python не остановит её ошибкой `RecursionError: maximum recursion depth exceeded`.

Ханойские башни

Есть три стержня и пирамидка из дисков на первом. Нужно перенести пирамидку на третий стержень, перекладывая по одному диску и никогда не кладя больший диск на меньший. Рекурсивное решение удивительно короткое:

PYTHON

```python
def hanoi(n, start, finish, middle):
    if n == 0:
        return
    hanoi(n - 1, start, middle, finish)       # убрать верхние n-1 дисков
    print(f"Диск {n}: {start} → {finish}")    # переложить самый большой
    hanoi(n - 1, middle, finish, start)       # положить n-1 дисков сверху

hanoi(3, "A", "C", "B")
```

РЕЗУЛЬТАТ

```
Диск 1: A → C
Диск 2: A → B
Диск 1: C → B
Диск 3: A → C
Диск 1: B → A
Диск 2: B → C
Диск 1: A → C
```

Числа Фибоначчи и мемоизация

В последовательности Фибоначчи каждое число — сумма двух предыдущих: 0, 1, 1, 2, 3, 5, 8, 13, ...

PYTHON

```python
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(10))
```

РЕЗУЛЬТАТ

```
55
```

Код красивый, но очень медленный: `fib(35)` считается несколько секунд, потому что одни и те же значения вычисляются тысячи раз. Решение — **мемоизация**: запоминать уже посчитанные ответы. В Python для этого есть готовый «декоратор» `lru_cache`:

```python
from functools import lru_cache

@lru_cache
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

print(fib(100))
```

РЕЗУЛЬТАТ

```
354224848179261915075
```

Теперь даже `fib(100)` считается мгновенно.

4.5 Быстрые сортировки

Сортировки из темы 4.3 работают за O(n²). Существуют алгоритмы, которые работают за O(n log n) — на больших данных это разница между секундами и часами. Оба основаны на идее **«разделяй и властвуй»**: разбить задачу на части, решить каждую рекурсивно и собрать ответ.

Сортировка слиянием

1. Делим список пополам.
2. Сортируем каждую половину (рекурсивно).
3. **Сливаем** две отсортированные половины в один отсортированный список.

```python
def merge_sort(items):
    if len(items) <= 1:
        return items
    middle = len(items) // 2
    left = merge_sort(items[:middle])
    right = merge_sort(items[middle:])

    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])     # добавить то, что осталось
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

РЕЗУЛЬТАТ

```
[3, 9, 10, 27, 38, 43, 82]
```

Быстрая сортировка

1. Выбираем **опорный** элемент.
2. Раскладываем остальные на три группы: меньше опорного, равные, больше.
3. Сортируем группы «меньше» и «больше» рекурсивно и склеиваем.

```python
def quick_sort(items):
    if len(items) <= 1:
        return items
    pivot = items[len(items) // 2]
    less = [x for x in items if x < pivot]
    equal = [x for x in items if x == pivot]
    greater = [x for x in items if x > pivot]
    return quick_sort(less) + equal + quick_sort(greater)

print(quick_sort([38, 27, 43, 3, 9, 82, 10]))
```

РЕЗУЛЬТАТ

```
[3, 9, 10, 27, 38, 43, 82]
```

Сравнение

Сортировка	Сложность	На 10 000 элементах (примерно)
Пузырьком	$O(n^2)$	5 с
Выбором	$O(n^2)$	2 с
Вставками	$O(n^2)$	2 с
Слиянием	O(n log n)	0,03 с
Быстрая	O(n log n)	0,02 с
`sorted()`	O(n log n)	0,001 с

Встроенная `sorted()` самая быстрая: она написана на языке C и использует улучшенный алгоритм Timsort. Поэтому в реальных программах всегда пользуются ею, а свои сортировки пишут для понимания и на олимпиадах.

4.6 Стек, очередь и поиск пути ДОП. ТЕМА

Стек

Стек — структура «последним пришёл — первым ушёл», как стопка тарелок: кладём сверху и берём сверху. В Python стек — это обычный список с методами `append` и `pop`.

Пример: проверка правильности скобок. Открывающую скобку кладём в стек, а встретив закрывающую — проверяем, что сверху лежит подходящая открывающая.

PYTHON

```python
def brackets_ok(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack

print(brackets_ok("(a[1] + {b})"))
print(brackets_ok("(]"))
```

РЕЗУЛЬТАТ

```
True
False
```

Очередь

Очередь — «первым пришёл — первым ушёл», как очередь в столовой. Для неё используют `deque` из модуля `collections`: он быстро добавляет в конец и забирает из начала.

```python
from collections import deque

queue = deque()
queue.append("Аня")
queue.append("Петя")
queue.append("Вика")
print(queue.popleft())   # первым обслуживают того, кто пришёл первым
print(list(queue))
```

РЕЗУЛЬТАТ

```
Аня
['Петя', 'Вика']
```

Поиск пути в лабиринте

Поиск в ширину (BFS) находит кратчайший путь. Идея: из старта проверяем всех соседей, потом соседей соседей и так далее — как круги на воде. Очередь хранит клетки, которые нужно проверить.

```python
from collections import deque

maze = [
    "S.#.....",
    ".##.###.",
    "....#...",
    ".##...#E",
]

def shortest_path(maze):
    rows, cols = len(maze), len(maze[0])
    start = (0, 0)
    queue = deque([(start, 0)])     # (клетка, сколько шагов до неё)
    visited = {start}
    while queue:
        (r, c), steps = queue.popleft()
        if maze[r][c] == "E":
            return steps
        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols \
                    and maze[nr][nc] != "#" and (nr, nc) not in visited:
                visited.add((nr, nc))
                queue.append(((nr, nc), steps + 1))
    return -1

print("Кратчайший путь:", shortest_path(maze), "шагов")
```

РЕЗУЛЬТАТ

```
Кратчайший путь: 12 шагов
```

Тот же алгоритм используют в играх, чтобы враги находили дорогу к игроку.

ЧАСТЬ 5

Файлы и данные

Пока программа работает, данные хранятся в переменных. Но стоит её закрыть — всё пропадает: рекорды, настройки, записи. Чтобы программа «помнила», данные нужно сохранять в файлы.

5.1 Текстовые файлы

Запись в файл

```python
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Первая строка\n")
    file.write("Вторая строка\n")
```

Разберём по частям:

- `open()` открывает файл. Первый аргумент — имя файла, второй — **режим**;
- `encoding="utf-8"` — кодировка, чтобы русские буквы сохранились правильно;
- `with ... as file:` — файл открывается, а после блока с отступом **автоматически закрывается**, даже если случилась ошибка;
- `\n` — переход на новую строку. Сам `write()` его не добавляет.

Режим	Что делает
`"r"`	чтение (по умолчанию). Если файла нет — ошибка `FileNotFoundError`
`"w"`	запись. Если файл есть — **стирает** его содержимое; если нет — создаёт
`"a"`	дозапись в конец. Старое содержимое сохраняется

> **Важно.** Режим `"w"` стирает файл целиком. Если нужно добавить строку к существующим — используй `"a"`.

Чтение файла

```python
with open("notes.txt", "r", encoding="utf-8") as file:
    text = file.read()        # весь файл одной строкой
print(text)
```

РЕЗУЛЬТАТ

```
Первая строка
Вторая строка
```

Чаще файл читают **построчно** — просто перебирая его в цикле:

```python
with open("notes.txt", encoding="utf-8") as file:
    for line in file:
        print(line.strip())   # strip убирает \n в конце строки
```

РЕЗУЛЬТАТ

```
Первая строка
Вторая строка
```

Ещё способ — `file.readlines()`: он вернёт список строк.

Пример: дневник

Программа дописывает запись с датой в конец файла:

```python
from datetime import date

entry = input("Что сегодня было? ")
with open("diary.txt", "a", encoding="utf-8") as file:
    file.write(f"{date.today()}: {entry}\n")
print("Записано!")
```

РЕЗУЛЬТАТ

```
Что сегодня было? Написал первую игру на Python
Записано!
```

Содержимое файла `diary.txt` после нескольких запусков:

```
2026-10-07: Разобрался со словарями
2026-10-08: Сделал виселицу
2026-10-09: Написал первую игру на Python
```

Если файла нет

```python
try:
    with open("records.txt", encoding="utf-8") as file:
        print(file.read())
except FileNotFoundError:
    print("Рекордов пока нет")
```

РЕЗУЛЬТАТ

```
Рекордов пока нет
```

5.2 Пути и папки

Если указать просто имя файла (`"notes.txt"`), Python ищет его в **текущей папке** — обычно это папка, из которой запущена программа. Чтобы работать с папками и путями, используют модуль `pathlib`.

```python
from pathlib import Path

folder = Path("saves")
folder.mkdir(exist_ok=True)          # создать папку, если её нет

file = folder / "game1.txt"          # путь saves/game1.txt
file.write_text("уровень 3", encoding="utf-8")

print(file.exists())                 # существует ли
print(file.read_text(encoding="utf-8"))
print(file.name, file.suffix)        # имя и расширение
```

РЕЗУЛЬТАТ

```
True
уровень 3
game1.txt .txt
```

Оператор `/` склеивает части пути правильно для любой системы: на Windows получится `saves\game1.txt`, на macOS — `saves/game1.txt`.

Перебор файлов в папке

```python
from pathlib import Path

for path in Path(".").iterdir():     # "." — текущая папка
    if path.is_file():
        print(path.name)
```

РЕЗУЛЬТАТ

```
main.py
notes.txt
diary.txt
```

Найти файлы по шаблону помогает `glob`: `Path(".").glob("*.py")` — все файлы с расширением `.py`.

5.3 CSV: таблицы

CSV (Comma-Separated Values) — простой формат таблиц: каждая строка файла — строка таблицы, значения разделены запятыми. CSV-файлы открывает Excel.

```
name,math,physics
Аня,5,4
Петя,4,4
Вика,5,5
```

Чтение CSV

Модуль `csv` с классом `DictReader` превращает каждую строку в словарь, где ключи — заголовки столбцов:

```python
import csv

with open("grades.csv", encoding="utf-8") as file:
    for row in csv.DictReader(file):
        avg = (int(row["math"]) + int(row["physics"])) / 2
        print(f"{row['name']}: средний балл {avg}")
```

РЕЗУЛЬТАТ

```
Аня: средний балл 4.5
Петя: средний балл 4.0
Вика: средний балл 5.0
```

Важно. Из CSV все значения читаются как **строки**. Числа нужно превращать через `int()` или `float()`.

Запись CSV

```python
import csv

records = [
    {"name": "Аня", "score": 120},
    {"name": "Петя", "score": 95},
]

with open("records.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=["name", "score"])
    writer.writeheader()        # строка с заголовками
    writer.writerows(records)   # все строки
```

Параметр `newline=""` нужен, чтобы на Windows между строками не появлялись пустые.

Совет. Если Excel показывает русские буквы «кракозябрами», сохраняй файл с кодировкой `encoding="utf-8-sig"`.

5.4 JSON

JSON — самый популярный формат для хранения и передачи данных. Он выглядит почти как словари и списки Python, поэтому работать с ним очень удобно.

```json
{
    "name": "Rex",
    "level": 5,
    "inventory": ["меч", "зелье"],
    "alive": true
}
```

JSON	Python
объект `{...}`	словарь
массив `[...]`	список
строка `"..."`	`str`
число	`int` или `float`
`true` / `false`	`True` / `False`
`null`	`None`

Сохранение

```python
import json

player = {
    "name": "Rex",
    "level": 5,
    "inventory": ["меч", "зелье"],
}

with open("save.json", "w", encoding="utf-8") as file:
    json.dump(player, file, ensure_ascii=False, indent=4)
```

- `ensure_ascii=False` — сохранять русские буквы как есть, а не кодами вида `м`;
- `indent=4` — красиво расставить отступы, чтобы файл было удобно читать.

Загрузка

```python
import json

with open("save.json", encoding="utf-8") as file:
    player = json.load(file)

print(player["name"], "уровень", player["level"])
print(player["inventory"])
```

РЕЗУЛЬТАТ

```
Rex уровень 5
['меч', 'зелье']
```

Пример: таблица рекордов

```python
import json
from pathlib import Path

RECORDS_FILE = Path("records.json")

def load_records():
    if not RECORDS_FILE.exists():
        return []
    with open(RECORDS_FILE, encoding="utf-8") as file:
        return json.load(file)

def save_record(name, score):
    records = load_records()
    records.append({"name": name, "score": score})
    records.sort(key=lambda r: r["score"], reverse=True)
    records = records[:5]                    # оставляем топ-5
    with open(RECORDS_FILE, "w", encoding="utf-8") as file:
        json.dump(records, file, ensure_ascii=False, indent=2)

save_record("Аня", 120)
save_record("Петя", 95)
for i, r in enumerate(load_records(), start=1):
    print(f"{i}. {r['name']} — {r['score']}")
```

РЕЗУЛЬТАТ

```
1. Аня — 120
2. Петя — 95
```

Такую пару функций «загрузить / сохранить» можно вставить в любую игру или программу.

> **Совет.** Метод `json.dumps(данные)` (с буквой `s` на конце) превращает данные не в файл, а в строку. Пригодится, когда данные нужно отправить по сети.

5.5 Дата и время

Для работы с датами есть модуль `datetime`.

```python
from datetime import date, datetime, timedelta

today = date.today()
print(today)
print(today.year, today.month, today.day)

now = datetime.now()
print(now.strftime("%d.%m.%Y %H:%M"))
```

РЕЗУЛЬТАТ

```
2026-10-09
2026 10 9
09.10.2026 18:30
```

Форматирование: strftime

`strftime` превращает дату в строку нужного вида. Коды формата:

Код	Значение	Пример
`%d`	день	`09`
`%m`	месяц	`10`
`%Y`	год	`2026`
`%H`	часы	`18`
`%M`	минуты	`30`
`%S`	секунды	`05`

Разбор строки: strptime

Обратная операция — из строки сделать дату:

```python
from datetime import datetime

birthday = datetime.strptime("15.03.2012", "%d.%m.%Y").date()
print(birthday)
```

РЕЗУЛЬТАТ

```
2012-03-15
```

Арифметика с датами

Даты можно вычитать — получится промежуток `timedelta`. А к дате можно прибавить промежуток.

```python
from datetime import date, timedelta

today = date(2026, 10, 9)
new_year = date(2027, 1, 1)
left = new_year - today
print(f"До Нового года осталось дней: {left.days}")

deadline = today + timedelta(days=14)
print("Сдать проект до", deadline.strftime("%d.%m.%Y"))
```

РЕЗУЛЬТАТ

```
До Нового года осталось дней: 84
Сдать проект до 23.10.2026
```

Даты можно сравнивать обычными `<`, `>`, `==`: более ранняя дата считается меньшей.

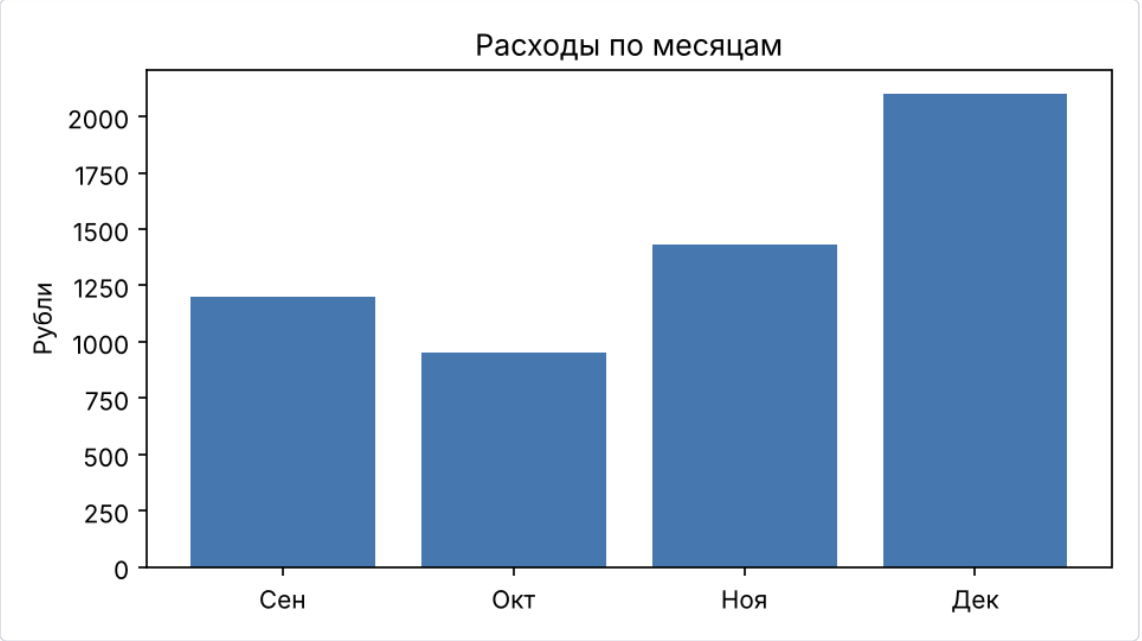
5.6 Графики в matplotlib ДОП. ТЕМА

Библиотека `matplotlib` рисует графики и диаграммы. Устанавливается командой `pip install matplotlib`.

```python
import matplotlib.pyplot as plt

months = ["Сен", "Окт", "Ноя", "Дек"]
spent = [1200, 950, 1430, 2100]

plt.bar(months, spent, color="#4a7bb0")
plt.title("Расходы по месяцам")
plt.ylabel("Рубли")
plt.savefig("expenses.png")   # сохранить картинку
plt.show()                    # показать окно с графиком
```



Столбчатая диаграмма расходов

Другие виды графиков:

- `plt.plot(x, y)` — линейный график (например, температура по дням);
- `plt.pie(values, labels=names)` — круговая диаграмма (доля трат по категориям);
- `plt.scatter(x, y)` — точечная диаграмма.

ЧАСТЬ 6

ООП

Объектно-ориентированное программирование (ООП) — способ организовать большую программу. Вместо сотен отдельных переменных и функций программа состоит из **объектов**: каждый объект сам хранит свои данные и умеет с ними работать. На ООП построены почти все большие программы и игры, включая игры на Pygame.

6.1 Классы и объекты

Представь, что в игре 50 врагов. У каждого есть имя, здоровье и сила атаки. Хранить это в отдельных переменных — `enemy1_hp`, `enemy2_hp`, ... — неудобно. Гораздо лучше описать **один раз**, что такое «враг», и создать столько врагов, сколько нужно.

- **Класс** — это чертёж, описание: какие у объекта есть данные и что он умеет.
- **Объект** (экземпляр) — конкретная вещь, созданная по чертежу.

Класс «Собака» — чертёж. Шарик и Бобик — объекты этого класса.

PYTHON

```python
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print(f"{self.name}: Гав!")

sharik = Dog("Шарик", 3)
bobik = Dog("Бобик", 5)

sharik.bark()
bobik.bark()
print(bobik.age)
```

РЕЗУЛЬТАТ

```
Шарик: Гав!
Бобик: Гав!
5
```

Разберём по частям:

- `class Dog:` — объявление класса. Имена классов пишут с заглавной буквы: `Dog`, `Player`, `BankAccount`.
- `__init__` — **конструктор**, специальный метод, который автоматически вызывается при создании объекта. В нём задают начальные данные.
- `self` — ссылка на **сам объект**. Через `self` объект обращается к своим данным: `self.name` — «моё имя».

- `self.name`, `self.age` — **атрибуты**, данные объекта.
- `bark` — **метод**, функция внутри класса. Первым параметром у любого метода идёт `self`.
- `Dog("Шарик", 3)` — создание объекта. Аргументы попадают в `__init__` (а `self` Python подставляет сам).
- `sharik.bark()` — вызов метода объекта.

> **Важно.** Не забывай `self` — и в параметрах метода, и при обращении к атрибутам. Если написать просто `name` вместо `self.name`, Python будет искать обычную переменную и выдаст `NameError`.

Методы меняют состояние объекта

PYTHON

```python
class Pet:
    def __init__(self, name):
        self.name = name
        self.hunger = 5      # от 0 (сыт) до 10 (очень голоден)
        self.mood = 5

    def feed(self):
        self.hunger = max(0, self.hunger - 3)
        print(f"{self.name} поел. Голод: {self.hunger}")

    def play(self):
        self.mood = min(10, self.mood + 2)
        self.hunger = min(10, self.hunger + 1)
        print(f"{self.name} поиграл. Настроение: {self.mood}")

    def status(self):
        print(f"{self.name}: голод {self.hunger}, настроение {self.mood}")

cat = Pet("Мурзик")
cat.play()
cat.play()
cat.feed()
cat.status()
```

РЕЗУЛЬТАТ

```
Мурзик поиграл. Настроение: 7
Мурзик поиграл. Настроение: 9
Мурзик поел. Голод: 4
Мурзик: голод 4, настроение 9
```

У каждого объекта свои данные. Если создать второго питомца, его голод и настроение будут независимыми.

6.2 Взаимодействие объектов

Объекты могут передаваться в методы других объектов и влиять друг на друга.

```python
class Hero:
    def __init__(self, name, hp, damage):
        self.name = name
        self.hp = hp
        self.damage = damage

    def attack(self, target):
        target.hp -= self.damage
        print(f"{self.name} атакует (урон {self.damage}). "
              f"{target.name}: {target.hp} hp")

    def is_alive(self):
        return self.hp > 0

knight = Hero("Рыцарь", 30, 8)
goblin = Hero("Гоблин", 20, 5)

while knight.is_alive() and goblin.is_alive():
    knight.attack(goblin)
    if goblin.is_alive():
        goblin.attack(knight)

winner = knight if knight.is_alive() else goblin
print(f"Победил {winner.name}!")
```

РЕЗУЛЬТАТ

```
Рыцарь атакует (урон 8). Гоблин: 12 hp
Гоблин атакует (урон 5). Рыцарь: 25 hp
Рыцарь атакует (урон 8). Гоблин: 4 hp
Гоблин атакует (урон 5). Рыцарь: 20 hp
Рыцарь атакует (урон 8). Гоблин: -4 hp
Победил Рыцарь!
```

Объекты можно хранить в списках, как любые другие значения:

```python
team = [Hero("Воин", 40, 6), Hero("Маг", 20, 12), Hero("Лучник", 25, 9)]
for hero in team:
    print(f"{hero.name}: {hero.hp} hp, урон {hero.damage}")
```

РЕЗУЛЬТАТ

```
Воин: 40 hp, урон 6
Маг: 20 hp, урон 12
Лучник: 25 hp, урон 9
```

6.3 Инкапсуляция и свойства

Инкапсуляция — это защита данных объекта от неправильного изменения. Представь банковский счёт: если любой код может написать `account.balance = -1000000`, ничего хорошего не выйдет.

Соглашение о подчёркивании

В Python атрибут, имя которого начинается с `_`, считается **внутренним**: «трогать снаружи не нужно». Технически доступ к нему есть, но это договорённость между программистами.

```python
class BankAccount:
    def __init__(self, owner):
        self.owner = owner
        self._balance = 0      # внутренний атрибут

    def deposit(self, amount):
        if amount <= 0:
            print("Сумма должна быть больше нуля")
            return
        self._balance += amount

    def withdraw(self, amount):
        if amount > self._balance:
            print("Недостаточно денег")
            return
        self._balance -= amount

    def get_balance(self):
        return self._balance

account = BankAccount("Аня")
account.deposit(500)
account.withdraw(800)
account.withdraw(200)
print(account.get_balance())
```

РЕЗУЛЬТАТ

```
Недостаточно денег
300
```

Деньги меняются только через методы, а методы проверяют, что операция допустима.

Свойства: @property

Писать `get_balance()` не очень удобно. **Свойство** позволяет обращаться к методу как к обычному атрибуту, а при изменении — проверять значение.

```python
class Player:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp            # здесь сработает сеттер с проверкой

    @property
    def hp(self):               # чтение: player.hp
        return self._hp

    @hp.setter
    def hp(self, value):        # запись: player.hp = ...
        self._hp = max(0, min(100, value))   # держим в пределах 0..100

p = Player("Rex", 80)
p.hp += 50
print(p.hp)
p.hp -= 500
print(p.hp)
```

PYTHON

РЕЗУЛЬТАТ

```
100
0
```

Снаружи `hp` выглядит как обычный атрибут, но здоровье никогда не станет больше 100 или меньше 0.

Свойство может быть и **вычисляемым** — без сеттера:

```python
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height

    @property
    def area(self):
        return self.width * self.height

r = Rectangle(4, 5)
print(r.area)
r.width = 10
print(r.area)
```

PYTHON

РЕЗУЛЬТАТ

```
20
50
```

6.4 Атрибуты класса, staticmethod и classmethod

Атрибуты класса

Атрибуты, созданные через `self` в `__init__`, у каждого объекта свои. А атрибуты, объявленные прямо в теле класса, — **общие** для всех объектов.

```python
class Enemy:
    count = 0                  # атрибут класса — один на всех

    def __init__(self, name):
        self.name = name       # атрибут объекта — у каждого свой
        Enemy.count += 1

Enemy("Гоблин")
Enemy("Орк")
Enemy("Тролль")
print("Врагов создано:", Enemy.count)
```

РЕЗУЛЬТАТ

```
Врагов создано: 3
```

Статические методы

`@staticmethod` — метод, которому не нужны данные объекта (у него нет `self`). Его кладут в класс просто потому, что он по смыслу к нему относится.

```python
class Temperature:
    @staticmethod
    def c_to_f(celsius):
        return celsius * 9 / 5 + 32

print(Temperature.c_to_f(100))
```

РЕЗУЛЬТАТ

```
212.0
```

Методы класса

`@classmethod` получает первым параметром не объект, а сам класс (`cls`). Чаще всего его используют как «альтернативный конструктор» — другой способ создать объект:

```python
class Player:
    def __init__(self, name, level):
        self.name = name
        self.level = level

    @classmethod
    def from_string(cls, text):
        name, level = text.split(":")
        return cls(name, int(level))

p = Player.from_string("Rex:12")
print(p.name, p.level)
```

РЕЗУЛЬТАТ

```
Rex 12
```

6.5 Магические методы

Магические методы — методы, имена которых начинаются и заканчиваются двумя подчёркиваниями: `__init__`, `__str__`, `__add__`. Python вызывает их сам в определённых ситуациях. С их помощью свои объекты можно научить работать с `print`, `+`, `==`, `len()` и сортировкой.

str: как объект выглядит при печати

Без `__str__` объект печатается непонятно:

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

print(Point(3, 4))
```

РЕЗУЛЬТАТ

```
<__main__.Point object at 0x7f1c2b3e4d90>
```

С `__str__` — так, как мы хотим:

```python
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

print(Point(3, 4))
```

РЕЗУЛЬТАТ

```
(3, 4)
```

Похожий метод `__repr__` используется, когда объект печатается внутри списка и в отладке.

Арифметика и сравнение

PYTHON

```python
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):           # v1 + v2
        return Vector(self.x + other.x, self.y + other.y)

    def __mul__(self, k):               # v * число
        return Vector(self.x * k, self.y * k)

    def __eq__(self, other):            # v1 == v2
        return self.x == other.x and self.y == other.y

    def __str__(self):
        return f"Vector({self.x}, {self.y})"

position = Vector(10, 5)
speed = Vector(2, -1)
print(position + speed)
print(speed * 3)
print(Vector(1, 1) == Vector(1, 1))
```

РЕЗУЛЬТАТ

```
Vector(12, 4)
Vector(6, -3)
True
```

Сортировка своих объектов

Чтобы список объектов можно было сортировать, достаточно определить `__lt__` — «меньше чем»:

PYTHON

```python
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def __lt__(self, other):
        return self.score < other.score

    def __repr__(self):
        return f"{self.name}({self.score})"

players = [Player("Аня", 120), Player("Петя", 95), Player("Вика", 150)]
print(sorted(players))
print(max(players))
```

РЕЗУЛЬТАТ

```
[Петя(95), Аня(120), Вика(150)]
Вика(150)
```

Метод	Когда вызывается
`__init__`	при создании объекта
`__str__`	`print(obj)`, `str(obj)`
`__repr__`	печать внутри списка, отладка
`__eq__`	`a == b`
`__lt__`	`a < b`, сортировка
`__add__`	`a + b`
`__len__`	`len(obj)`
`__contains__`	`x in obj`

6.6 Наследование

Наследование позволяет создать новый класс на основе существующего. Новый (дочерний) класс получает все атрибуты и методы родительского и может добавить свои или изменить старые.

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} ест")

    def speak(self):
        print(f"{self.name} издаёт звук")

class Cat(Animal):          # Cat наследует от Animal
    def speak(self):        # переопределяем метод
        print(f"{self.name}: Мяу!")

class Dog(Animal):
    def speak(self):
        print(f"{self.name}: Гав!")

    def fetch(self):        # новый метод, которого нет у Animal
        print(f"{self.name} принёс палку")

cat = Cat("Мурзик")
dog = Dog("Шарик")
cat.eat()          # метод родителя
cat.speak()        # свой метод
dog.speak()
dog.fetch()
```

РЕЗУЛЬТАТ

```
Мурзик ест
Мурзик: Мяу!
Шарик: Гав!
Шарик принёс палку
```

Наследование отвечает на вопрос **«является ли»**: кошка *является* животным, маг *является* персонажем. Если так сказать нельзя — наследование не подходит.

super(): вызов метода родителя

Если у дочернего класса свой `__init__`, нужно не забыть вызвать `__init__` родителя, иначе атрибуты родителя не создадутся. Для этого есть `super()`:

```python
class Character:
    def __init__(self, name, hp):
        self.name = name
        self.hp = hp

class Mage(Character):
    def __init__(self, name, hp, mana):
        super().__init__(name, hp)   # родитель задаёт name и hp
        self.mana = mana             # а это — только у мага

m = Mage("Мерлин", 60, 100)
print(m.name, m.hp, m.mana)
```

РЕЗУЛЬТАТ

```
Мерлин 60 100
```

`super()` можно вызывать и в обычных методах, чтобы **дополнить** поведение родителя, а не заменить его целиком:

```python
class Character:
    def __init__(self, name):
        self.name = name

    def describe(self):
        print(f"Персонаж: {self.name}")

class Archer(Character):
    def describe(self):
        super().describe()           # сначала то, что делает родитель
        print("Класс: лучник, стреляет издалека")

Archer("Робин").describe()
```

РЕЗУЛЬТАТ

```
Персонаж: Робин
Класс: лучник, стреляет издалека
```

isinstance: проверка типа

```python
print(isinstance(cat, Cat))
print(isinstance(cat, Animal))   # кошка — тоже животное
print(isinstance(cat, Dog))
```

РЕЗУЛЬТАТ

```
True
True
False
```

Совет. В Pygame наследование используется постоянно: каждый игровой объект — игрок, враг, пуля — это класс, унаследованный от `pygame.sprite.Sprite`.

6.7 Полиморфизм и абстрактные классы

Полиморфизм

Полиморфизм — когда у разных объектов есть метод с одинаковым именем, и каждый выполняет его по-своему. Код, который вызывает этот метод, не обязан знать, с каким именно объектом работает.

PYTHON

```python
class Circle:
    def __init__(self, r):
        self.r = r

    def area(self):
        return 3.14 * self.r ** 2

class Square:
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

shapes = [Circle(2), Square(3), Circle(1)]
for shape in shapes:
    print(type(shape).__name__, shape.area())
```

РЕЗУЛЬТАТ

```
Circle 12.56
Square 9
Circle 3.14
```

Цикл просто вызывает `shape.area()`, а нужную формулу выбирает сам объект. Добавить новую фигуру можно, не трогая цикл.

Абстрактные классы

Иногда родительский класс — это лишь общая идея, и создавать его объекты не нужно. Например, «фигура вообще» не имеет площади, площадь есть только у конкретных фигур. Такой класс делают **абстрактным** с помощью модуля `abc`:

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

print(Square(4).area())
shape = Shape()
```

РЕЗУЛЬТАТ

```
16
TypeError: Can't instantiate abstract class Shape without an implementation for abstract
method 'area'
```

Абстрактный метод — это обязательство: каждый дочерний класс **должен** его реализовать. Если забыть, Python сообщит об ошибке сразу, при создании объекта.

6.8 Композиция и dataclass

Композиция: «содержит»

Не всё нужно решать наследованием. Часто объект просто **содержит** другие объекты. У героя *есть* инвентарь, а не герой *является* инвентарём.

PYTHON

```python
class Item:
    def __init__(self, name, weight):
        self.name = name
        self.weight = weight

class Inventory:
    def __init__(self, max_weight):
        self.items = []
        self.max_weight = max_weight

    def add(self, item):
        if self.total_weight() + item.weight > self.max_weight:
            print(f"{item.name} не помещается!")
            return
        self.items.append(item)

    def total_weight(self):
        return sum(item.weight for item in self.items)

class Hero:
    def __init__(self, name):
        self.name = name
        self.inventory = Inventory(max_weight=10)   # герой СОДЕРЖИТ инвентарь

hero = Hero("Rex")
hero.inventory.add(Item("Меч", 6))
hero.inventory.add(Item("Щит", 5))
hero.inventory.add(Item("Зелье", 1))
print([item.name for item in hero.inventory.items])
```

РЕЗУЛЬТАТ

```
Щит не помещается!
['Меч', 'Зелье']
```

Совет. Правило выбора: «является» — наследование, «содержит» или «имеет» — композиция. Если сомневаешься, выбирай композицию: её проще менять.

dataclass: классы для хранения данных

Если класс нужен в основном для того, чтобы хранить данные, `__init__`, `__repr__` и `__eq__` можно не писать вручную. Это сделает декоратор `@dataclass`:

```python
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    year: int
    read: bool = False

book = Book("Гарри Поттер", "Дж. Роулинг", 1997)
print(book)
print(book.title)
print(book == Book("Гарри Поттер", "Дж. Роулинг", 1997))
```

PYTHON

РЕЗУЛЬТАТ

```
Book(title='Гарри Поттер', author='Дж. Роулинг', year=1997, read=False)
Гарри Поттер
True
```

Записи `title: str` — это **аннотации типов**: подсказки, какого типа должно быть значение. Python их не проверяет, но редактор использует их для подсказок.

Как разбить задачу на классы

1. Выпиши **существительные** из описания задачи: герой, враг, предмет, инвентарь, карта. Это кандидаты в классы.
2. Для каждого класса выпиши, **что он знает** (атрибуты) и **что умеет** (методы).
3. Определи связи: кто от кого наследуется, кто кого содержит.
4. Нарисуй схему на бумаге **до** того, как писать код.

```
Character  ——— содержит ———►  Inventory
name, hp                      items, max_weight
attack(), is_alive()          add(), total_weight()
   ▲            ▲
   | является    | является
Warrior        Mage
rage           mana
```

ЧАСТЬ 7

Pygame

Pygame — библиотека для создания 2D-игр на Python. Всё, что ты уже знаешь — переменные, циклы, условия, списки, функции и классы, — здесь складывается в настоящие игры с графикой, управлением и звуком.

7.1 Окно и игровой цикл

Установка

ТЕРМИНАЛ

```
pip install pygame-ce
```

`pygame-ce` — это активно развивающаяся версия Pygame. Подключается она так же: `import pygame`. Подойдёт и обычный `pip install pygame`.

Самая простая игра: пустое окно

PYTHON

```python
import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))   # окно 800 × 600 пикселей
pygame.display.set_caption("Моя игра")
clock = pygame.time.Clock()

running = True
while running:
    # 1. События: что сделал игрок
    for event in pygame.event.get():
        if event.type == pygame.QUIT:          # нажали крестик окна
            running = False

    # 2. Обновление: двигаем объекты, считаем очки
    # (пока ничего)

    # 3. Отрисовка: рисуем кадр
    screen.fill((30, 30, 40))                  # залить фон
    pygame.display.flip()                      # показать кадр на экране

    clock.tick(60)                             # не больше 60 кадров в секунду

pygame.quit()
```

Игровой цикл

Любая игра — это цикл, который повторяется много раз в секунду. Один проход цикла — один **кадр**. В каждом кадре три шага:

1. **События** — проверить, что сделал игрок: нажал клавишу, кликнул мышью, закрыл окно.
2. **Обновление** — сдвинуть объекты, проверить столкновения, посчитать очки.
3. **Отрисовка** — очистить экран и нарисовать всё заново в новых позициях.

Мультфильм тоже состоит из кадров: если быстро показывать картинки, где персонаж немного сдвинулся, кажется, что он движется. Игра делает то же самое 60 раз в секунду.

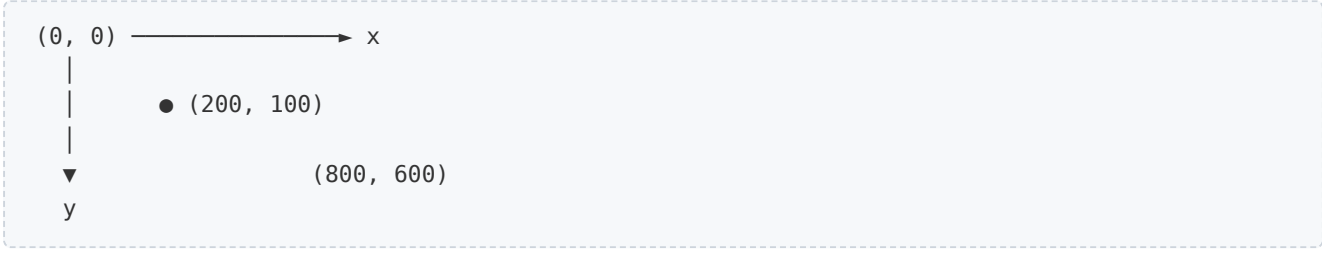
- `pygame.event.get()` — список всех событий, которые случились с прошлого кадра;
- `screen.fill(цвет)` — залить весь экран цветом (стереть прошлый кадр);
- `pygame.display.flip()` — показать нарисованный кадр. Всё рисуется сначала «в памяти», а `flip()` выводит готовую картинку целиком;
- `clock.tick(60)` — притормозить цикл так, чтобы было не больше 60 кадров в секунду (**FPS**, frames per second). Без этого игра на разных компьютерах шла бы с разной скоростью.

> **Важно.** Не забывай обрабатывать событие `pygame.QUIT`. Без него окно невозможно будет закрыть крестиком.

Цвета и координаты

Цвет в Pygame задаётся тремя числами от 0 до 255 — сколько в нём **красного, зелёного и синего** (RGB): `(255, 0, 0)` — красный, `(0, 0, 0)` — чёрный, `(255, 255, 255)` — белый, `(255, 200, 0)` — жёлтый.

Координаты отсчитываются от **левого верхнего угла** окна. Ось X идёт вправо, а ось **Y — вниз** (не вверх, как в математике).



7.2 Рисование фигур

Модуль `pygame.draw` рисует простые фигуры. Первым аргументом всегда идёт поверхность, на которой рисуем (`screen`), вторым — цвет.

Функция	Что рисует
`pygame.draw.rect(screen, color, (x, y, w, h))`	прямоугольник: левый верхний угол, ширина, высота
`pygame.draw.circle(screen, color, (x, y), r)`	круг: центр и радиус
`pygame.draw.line(screen, color, (x1, y1), (x2, y2), width)`	линия
`pygame.draw.polygon(screen, color, [(x1, y1), (x2, y2), ...])`	многоугольник по точкам
`pygame.draw.ellipse(screen, color, (x, y, w, h))`	эллипс, вписанный в прямоугольник

Если в конце добавить толщину линии (например, `3`), фигура будет нарисована только контуром.

PYTHON

```python
import pygame

pygame.init()
screen = pygame.display.set_mode((600, 400))
clock = pygame.time.Clock()

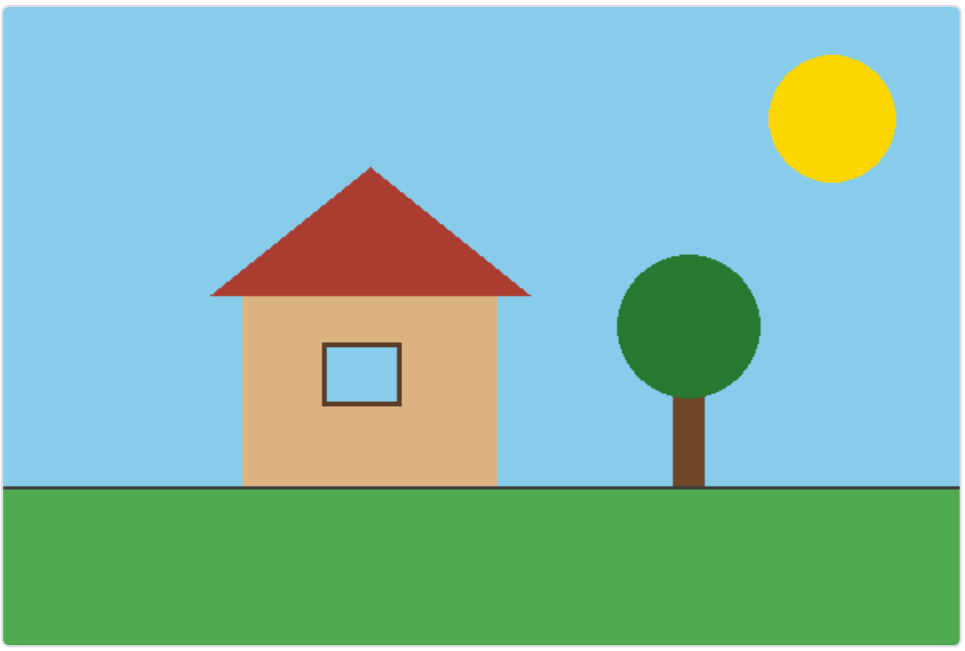
SKY = (135, 206, 235)
GRASS = (80, 170, 80)
WALL = (220, 180, 130)
ROOF = (170, 60, 50)
SUN = (255, 215, 0)

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill(SKY)
    pygame.draw.rect(screen, GRASS, (0, 300, 600, 100))           # трава
    pygame.draw.circle(screen, SUN, (520, 70), 40)                # солнце
    pygame.draw.rect(screen, WALL, (150, 180, 160, 120))          # стена дома
    pygame.draw.polygon(screen, ROOF, [(130, 180), (230, 100), (330, 180)])  # крыша
    pygame.draw.rect(screen, SKY, (200, 210, 50, 40))             # окно
    pygame.draw.rect(screen, (90, 60, 40), (200, 210, 50, 40), 3)  # рамка окна
    pygame.draw.rect(screen, (110, 70, 40), (420, 220, 20, 80))    # ствол
    pygame.draw.circle(screen, (40, 120, 50), (430, 200), 45)      # крона
    pygame.draw.line(screen, (60, 60, 60), (0, 300), (600, 300), 2)

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Картинка из фигур

> **Совет.** Фигуры рисуются по порядку: то, что нарисовано позже, оказывается **поверх**. Поэтому сначала рисуют фон, потом объекты.

7.3 Движение и управление с клавиатуры

Движение

Чтобы объект двигался, его координаты нужно немного менять **в каждом кадре**. Пример: мяч, который отскакивает от стен.

```python
import pygame

pygame.init()
WIDTH, HEIGHT = 600, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

x, y = 100, 100          # позиция мяча
dx, dy = 4, 3            # скорость: на сколько пикселей сдвигаться за кадр
radius = 20

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    x += dx
    y += dy
    if x - radius < 0 or x + radius > WIDTH:    # удар о левую или правую стену
        dx = -dx
```

```
    if y - radius < 0 or y + radius > HEIGHT:   # удар о верх или низ
        dy = -dy

    screen.fill((20, 24, 35))
    pygame.draw.circle(screen, (255, 120, 80), (x, y), radius)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Мяч, отскакивающий от стен

`dx` и `dy` — скорость мяча по горизонтали и вертикали. Когда мяч касается стены, мы меняем знак скорости, и мяч летит в обратную сторону.

Прямоугольники: pygame.Rect

Положение и размер объектов в Pygame удобно хранить в `pygame.Rect`. У прямоугольника много полезных атрибутов:

```
          top
   topleft ┌──────┐ topright
      left │center│ right
bottomleft └──────┘ bottomright
          bottom
```

```python
import pygame

player = pygame.Rect(100, 50, 40, 40)   # x, y, ширина, высота
print(player.x, player.y)
print(player.right, player.bottom)
print(player.center)
player.center = (300, 200)              # переместить по центру
print(player.topleft)
```

РЕЗУЛЬТАТ

```
100 50
140 90
(120, 70)
(280, 180)
```

Управление с клавиатуры

Есть два способа узнать о нажатиях:

- **событие** `KEYDOWN` — срабатывает один раз в момент нажатия. Подходит для выстрела, прыжка, паузы;
- `pygame.key.get_pressed()` — показывает, какие клавиши **зажаты прямо сейчас**. Подходит для плавного движения.

```python
import pygame

pygame.init()
WIDTH, HEIGHT = 600, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

player = pygame.Rect(280, 180, 40, 40)   # прямоугольник: x, y, ширина, высота
speed = 5

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            running = False                 # выход по Esc

    keys = pygame.key.get_pressed()         # какие клавиши зажаты прямо сейчас
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    if keys[pygame.K_UP]:
        player.y -= speed
    if keys[pygame.K_DOWN]:
        player.y += speed

    player.clamp_ip(screen.get_rect())      # не выпускать за край экрана
```

```
    screen.fill((20, 24, 35))
    pygame.draw.rect(screen, (90, 200, 120), player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Квадрат, управляемый стрелками

Клавиша	Код
стрелки	`pygame.K_LEFT`, `K_RIGHT`, `K_UP`, `K_DOWN`
буквы	`pygame.K_a`, `K_w`, `K_s`, `K_d`, …
пробел	`pygame.K_SPACE`
Enter	`pygame.K_RETURN`
Esc	`pygame.K_ESCAPE`

`player.clamp_ip(screen.get_rect())` не даёт квадрату выйти за пределы экрана.

> **Совет.** Чтобы скорость движения не зависела от FPS, используют время кадра: `dt = clock.tick(60) / 1000` — сколько секунд прошло с прошлого кадра. Тогда движение пишут как `x += speed * dt`, где `speed` — пикселей в секунду.

7.4 Мышь и текст

Мышь

Что нужно	Как получить
позиция курсора	`pygame.mouse.get_pos()` → `(x, y)`
зажаты ли кнопки	`pygame.mouse.get_pressed()` → `(левая, средняя, правая)`
клик	событие `MOUSEBUTTONDOWN`, в нём `event.pos` и `event.button` (1 — левая, 3 — правая)
движение	событие `MOUSEMOTION`

Текст

Текст в Pygame — это картинка. Сначала создаётся шрифт, потом он «рендерит» строку в картинку, а картинка выводится на экран методом `blit`.

```python
font = pygame.font.Font(None, 40)                    # None — стандартный шрифт,
40 — размер
text = font.render("Привет!", True, (255, 255, 255))  # текст, сглаживание, цвет
screen.blit(text, (20, 20))                          # нарисовать в точке (20, 20)
```

Пример: «Поймай мишень». Мишень прыгает в случайное место после каждого попадания, а счёт выводится в углу.

```python
import random
import pygame

pygame.init()
WIDTH, HEIGHT = 600, 400
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
font = pygame.font.Font(None, 40)

target = pygame.Vector2(300, 200)   # центр мишени
radius = 30
score = 0

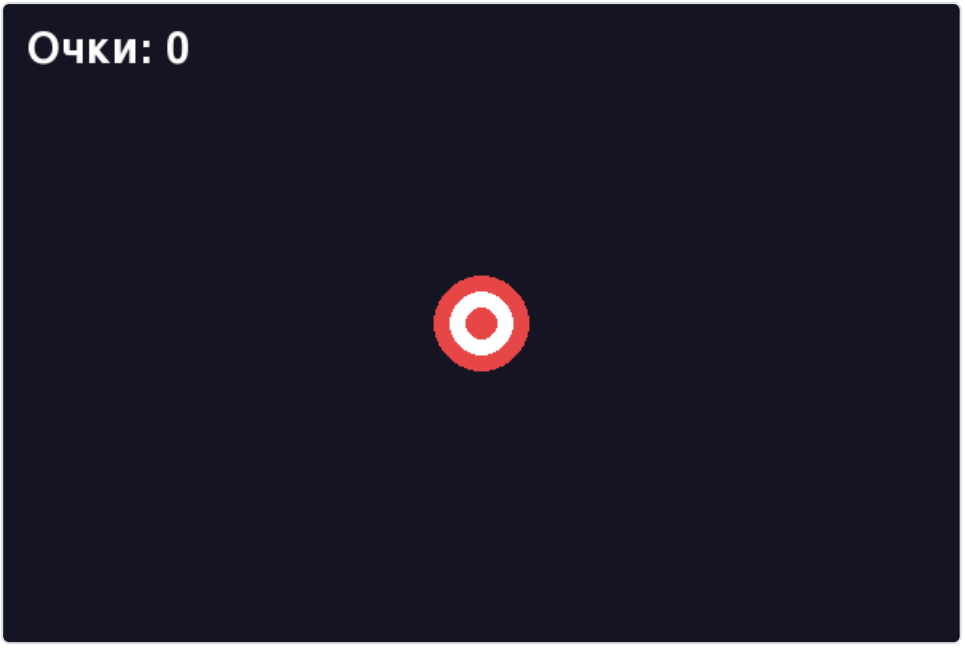
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if target.distance_to(event.pos) <= radius:   # попали по мишени
                score += 1
                target.x = random.randint(radius, WIDTH - radius)
                target.y = random.randint(60, HEIGHT - radius)

    screen.fill((20, 24, 35))
    pygame.draw.circle(screen, (230, 70, 70), target, radius)
    pygame.draw.circle(screen, (255, 255, 255), target, radius - 10)
    pygame.draw.circle(screen, (230, 70, 70), target, radius - 20)
```

```
    text = font.render(f"Очки: {score}", True, (255, 255, 255))
    screen.blit(text, (15, 15))                    # вывести текст на экран

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Игра «Поймай мишень»

`pygame.Vector2` — точка или вектор на плоскости. Метод `distance_to` считает расстояние до другой точки: если клик ближе радиуса — попали.

7.5 Картинки и звук

Нарисованные фигуры — это хорошо, но в настоящих играх используют картинки (**спрайты**) и звуки.

Картинки

```python
player_image = pygame.image.load("images/player.png").convert_alpha()
player_image = pygame.transform.scale(player_image, (64, 64))   # изменить размер

# в цикле отрисовки:
screen.blit(player_image, (x, y))
```

- `pygame.image.load` загружает файл (PNG, JPG);
- `convert_alpha()` ускоряет отрисовку и сохраняет прозрачность PNG. Вызывать его нужно **после** `set_mode`;

- `pygame.transform.flip(image, True, False)` отражает картинку по горизонтали — удобно, когда персонаж поворачивается;
- `pygame.transform.rotate(image, 90)` поворачивает картинку.

Узнать прямоугольник картинки и сразу поставить его в нужное место:

```python
rect = player_image.get_rect(center=(400, 300))
screen.blit(player_image, rect)
```

Звук и музыка

```python
pygame.mixer.init()
jump_sound = pygame.mixer.Sound("sounds/jump.wav")
jump_sound.play()                          # проиграть звук один раз

pygame.mixer.music.load("sounds/music.mp3")
pygame.mixer.music.set_volume(0.5)         # громкость от 0 до 1
pygame.mixer.music.play(-1)                # -1 — повторять бесконечно
```

`Sound` — для коротких эффектов (прыжок, выстрел, монетка), `music` — для фоновой музыки.

Где брать графику и звуки

Бесплатные наборы для игр есть на сайтах **Kenney** (kenney.nl) и **OpenGameArt** (opengameart.org). Перед использованием посмотри **лицензию**: можно ли брать для своих проектов и нужно ли указывать автора.

> **Совет.** Держи файлы в порядке: картинки в папке `images`, звуки — в `sounds`. И указывай пути относительно главного файла игры.

7.6 Спрайты и группы

Когда объектов в игре становится много — враги, пули, монеты, — удобно сделать каждый из них **спрайтом**: объектом класса, унаследованного от `pygame.sprite.Sprite`. Тут пригодится всё, что ты знаешь про ООП.

У каждого спрайта должны быть два атрибута:

- `self.image` — как он выглядит (картинка или нарисованная поверхность);
- `self.rect` — где он находится и какого размера.

И обычно метод `update()` — что спрайт делает в каждом кадре.

```python
class Coin(pygame.sprite.Sprite):
    def __init__(self, x, y):
        super().__init__()
        self.image = pygame.image.load("images/coin.png").convert_alpha()
        self.rect = self.image.get_rect(center=(x, y))

    def update(self):
        self.rect.y += 2       # монета медленно падает
```

Группы спрайтов

Группа (`pygame.sprite.Group`) хранит много спрайтов и умеет работать со всеми сразу:

- `group.update()` — вызвать `update()` у каждого спрайта;
- `group.draw(screen)` — нарисовать все спрайты;
- `group.add(sprite)` — добавить спрайт;
- `sprite.kill()` — удалить спрайт из всех групп, в которых он состоит.

Обычно делают одну общую группу `all_sprites` для обновления и отрисовки и отдельные группы по видам объектов (`enemies`, `bullets`, `coins`) для проверки столкновений.

7.7 Столкновения

Способ	Что проверяет
`rect1.colliderect(rect2)`	пересекаются ли два прямоугольника
`rect.collidepoint(x, y)`	попала ли точка (например, клик) в прямоугольник
`rect.collidelist(список)`	с каким прямоугольником из списка есть столкновение (`-1`, если ни с каким)
`pygame.sprite.spritecollide(sprite, group, dokill)`	с какими спрайтами группы столкнулся спрайт; если `dokill=True` — удалить их
`pygame.sprite.groupcollide(g1, g2, kill1, kill2)`	все столкновения между двумя группами (пули и враги)

Соберём всё вместе: игра «Поймай фрукты». Корзина управляется стрелками, фрукты падают сверху, пойманные исчезают и прибавляют очки.

```python
import random
import pygame

WIDTH, HEIGHT = 600, 400


class Basket(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((90, 20))
```

```
        self.image.fill((240, 190, 80))
        self.rect = self.image.get_rect(midbottom=(WIDTH // 2, HEIGHT - 10))
        self.speed = 7

    def update(self):
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            self.rect.x -= self.speed
        if keys[pygame.K_RIGHT]:
            self.rect.x += self.speed
        self.rect.clamp_ip(pygame.Rect(0, 0, WIDTH, HEIGHT))


class Fruit(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = pygame.Surface((24, 24), pygame.SRCALPHA)   # прозрачный фон
        pygame.draw.circle(self.image, (230, 70, 70), (12, 12), 12)
        x = random.randint(20, WIDTH - 20)
        self.rect = self.image.get_rect(center=(x, -20))
        self.speed = random.randint(2, 5)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > HEIGHT:     # улетел за экран
            self.kill()                # удалить спрайт из всех групп


pygame.init()
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
font = pygame.font.Font(None, 36)

basket = Basket()
fruits = pygame.sprite.Group()
all_sprites = pygame.sprite.Group(basket)
score = 0
spawn_timer = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

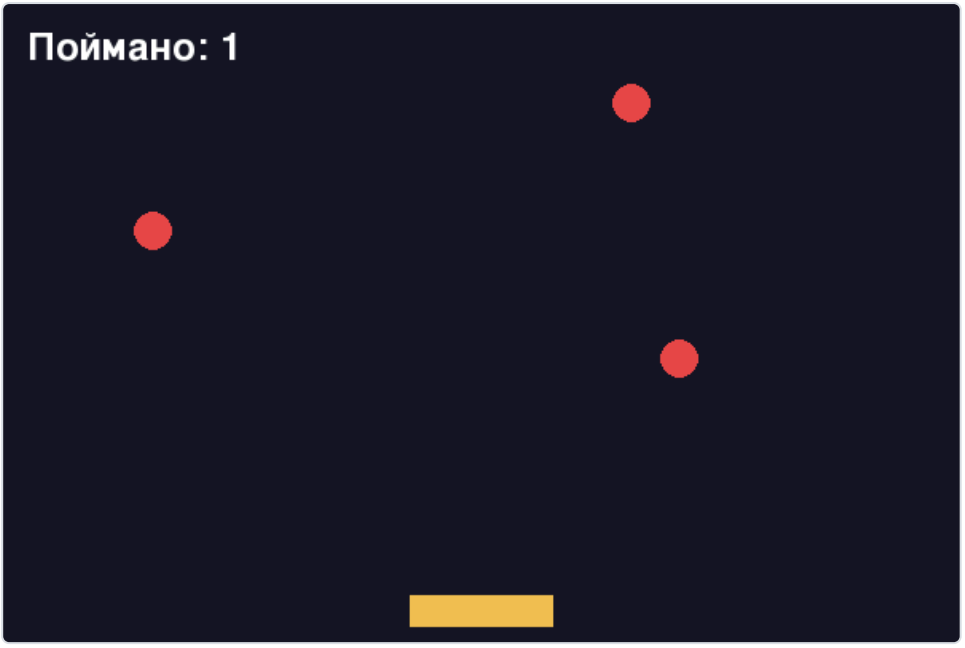
    spawn_timer += 1
    if spawn_timer >= 40:              # новый фрукт примерно раз в 2/3 секунды
        spawn_timer = 0
        fruit = Fruit()
        fruits.add(fruit)
        all_sprites.add(fruit)

    all_sprites.update()               # вызвать update() у каждого спрайта

    caught = pygame.sprite.spritecollide(basket, fruits, True)   # True — удалить пойманные
    score += len(caught)
```

```
    screen.fill((20, 24, 35))
    all_sprites.draw(screen)           # нарисовать все спрайты
    screen.blit(font.render(f"Поймано: {score}", True, (255, 255, 255)), (15, 15))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



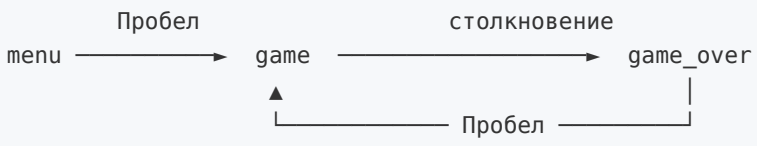
Игра «Поймай фрукты»

Обрати внимание, как устроен код: у каждого класса своя задача. `Basket` знает, как двигаться по нажатию клавиш, `Fruit` — как падать и исчезать за экраном. Главный цикл только создаёт фрукты, вызывает `update()` у всех и проверяет столкновения.

> **Совет.** Столкновения проверяются по прямоугольникам. Если картинка с большими прозрачными краями, столкновение будет срабатывать «издалека». Для точных столкновений есть `pygame.sprite.collide_mask`.

7.8 Состояния игры

В настоящей игре есть не только сам игровой процесс, но и **меню**, **пауза**, экран **«Игра окончена»**. Каждый такой экран — это **состояние** игры. Удобно хранить текущее состояние в переменной и в цикле рисовать то, что нужно для этого состояния.




```python
import random
import pygame
```

```
WIDTH, HEIGHT = 600, 400
WHITE = (255, 255, 255)

pygame.init()
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()
big_font = pygame.font.Font(None, 72)
font = pygame.font.Font(None, 36)


def draw_center(text, font, y):
    """Нарисовать текст по центру экрана по горизонтали."""
    surface = font.render(text, True, WHITE)
    screen.blit(surface, surface.get_rect(center=(WIDTH // 2, y)))


def new_game():
    """Вернуть начальные значения для новой игры."""
    return pygame.Rect(WIDTH // 2 - 20, HEIGHT - 60, 40, 40), [], 0


state = "menu"            # "menu", "game" или "game_over"
best = 0
player, enemies, score = new_game()

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if state in ("menu", "game_over"):
                player, enemies, score = new_game()
                state = "game"

    screen.fill((20, 24, 35))

    if state == "menu":
        draw_center("УВЕРНИСЬ!", big_font, 150)
        draw_center("Стрелки — движение", font, 230)
        draw_center("Пробел — начать игру", font, 270)

    elif state == "game":
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LEFT]:
            player.x -= 6
        if keys[pygame.K_RIGHT]:
            player.x += 6
        player.clamp_ip(screen.get_rect())

        if random.random() < 0.05:                    # иногда появляется враг
            enemies.append(pygame.Rect(random.randint(0, WIDTH - 30), -30, 30, 30))
        for enemy in enemies:
            enemy.y += 5
        enemies = [e for e in enemies if e.top < HEIGHT]
        score += 1
```

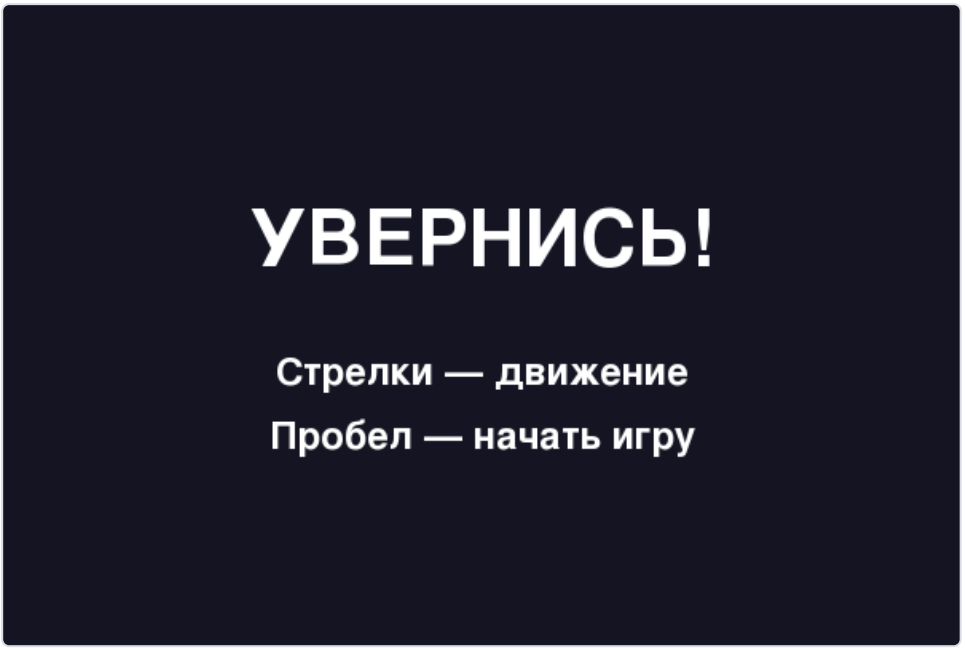
```
        if player.collidelist(enemies) != -1:        # столкнулся с любым врагом
            best = max(best, score)
            state = "game_over"

        pygame.draw.rect(screen, (90, 200, 120), player)
        for enemy in enemies:
            pygame.draw.rect(screen, (230, 70, 70), enemy)
        screen.blit(font.render(f"Очки: {score}", True, WHITE), (15, 15))

    elif state == "game_over":
        draw_center("ИГРА ОКОНЧЕНА", big_font, 150)
        draw_center(f"Очки: {score}   Рекорд: {best}", font, 220)
        draw_center("Пробел — сыграть ещё", font, 270)

    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Меню игры «Уверните!»

Функция `new_game()` возвращает всё в начальное состояние: так игру легко перезапустить, не перезапуская программу.

Рекорд между запусками

Чтобы рекорд не пропадал после закрытия игры, его сохраняют в файл — например, в JSON, как в теме 5.4:

```python
import json
from pathlib import Path

BEST_FILE = Path("best.json")

def load_best():
    if BEST_FILE.exists():
        return json.loads(BEST_FILE.read_text(encoding="utf-8"))["best"]
    return 0

def save_best(best):
    BEST_FILE.write_text(json.dumps({"best": best}), encoding="utf-8")
```

Как организовать большую игру

Когда игра растёт, код раскладывают по файлам:

```
my_game/
├── main.py          ← запуск и главный цикл
├── settings.py      ← размеры экрана, цвета, скорость
├── player.py        ← класс игрока
├── enemies.py       ← классы врагов
├── levels.py        ← загрузка уровней
├── images/
└── sounds/
```

Уровни из текстового файла

Уровни удобно «рисовать» символами в текстовом файле: `#` — стена, `.` — пусто, `*` — монета, `P` — игрок. Программа читает файл и создаёт объекты:

```
##########
#P...*...#
#.####...#
#....*..*#
##########
```

```python
TILE = 40

with open("level1.txt", encoding="utf-8") as file:
    rows = [line.rstrip("\n") for line in file]

walls = []
for row_index, row in enumerate(rows):
    for col_index, symbol in enumerate(row):
        x, y = col_index * TILE, row_index * TILE
        if symbol == "#":
            walls.append(pygame.Rect(x, y, TILE, TILE))
```

Новый уровень — это просто новый текстовый файл, код менять не нужно.

7.9 Прыжки и гравитация ДОП. ТЕМА

В платформерах у персонажа есть **вертикальная скорость**. Гравитация каждый кадр увеличивает её (тянет вниз), а прыжок задаёт отрицательную скорость (вверх). Когда персонаж падает на платформу, он встаёт на неё, и скорость обнуляется.

PYTHON

```python
import pygame

WIDTH, HEIGHT = 600, 400
GRAVITY = 0.6
JUMP_SPEED = -12

pygame.init()
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

player = pygame.Rect(100, 300, 30, 40)
y = float(player.y)      # точная координата по вертикали (Rect хранит только целые)
velocity_y = 0           # вертикальная скорость
on_ground = False

platforms = [
    pygame.Rect(0, 360, 600, 40),      # земля
    pygame.Rect(180, 280, 120, 15),
    pygame.Rect(340, 210, 120, 15),
    pygame.Rect(480, 140, 100, 15),
]

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE and on_ground:
            velocity_y = JUMP_SPEED          # прыжок: скорость вверх

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= 5
    if keys[pygame.K_RIGHT]:
        player.x += 5

    velocity_y += GRAVITY                    # гравитация тянет вниз
    y += velocity_y
    player.y = round(y)

    on_ground = False
    for platform in platforms:
        # падаем сверху на платформу — встаём на неё
        if player.colliderect(platform) and velocity_y > 0:
            player.bottom = platform.top
            y = player.y
            velocity_y = 0
            on_ground = True
```

```
    screen.fill((25, 30, 45))
    for platform in platforms:
        pygame.draw.rect(screen, (120, 120, 140), platform)
    pygame.draw.rect(screen, (255, 200, 80), player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```



Прыжок по платформам

Координата `y` хранится в отдельной дробной переменной, потому что `Rect` хранит только целые числа, а гравитация добавляет к скорости дробные 0.6.

Анимация

Анимация спрайта — это несколько картинок (**кадров**), которые сменяют друг друга:

PYTHON

```
frames = [pygame.image.load(f"images/run{i}.png").convert_alpha() for i in
range(4)]
frame_index = 0

# в update():
frame_index = (frame_index + 0.15) % len(frames)   # 0.15 — скорость анимации
self.image = frames[int(frame_index)]
```

Сборка игры в .exe

Чтобы друг мог запустить игру без установки Python, её собирают в один файл программой PyInstaller:

ТЕРМИНАЛ

```
pip install pyinstaller
pyinstaller --onefile --windowed main.py
```

Готовый файл появится в папке `dist`. Папки с картинками и звуками положи рядом с ним.

ЧАСТЬ 8

Python для реальных задач

В этой части программы выходят за пределы компьютера: получают данные из интернета, хранят их в настоящей базе данных и общаются с людьми через мессенджер.

8.1 Данные из интернета

Как устроен запрос

Когда ты открываешь сайт, браузер (**клиент**) отправляет **запрос** на другой компьютер (**сервер**), а сервер присылает **ответ**. Программа на Python умеет делать то же самое.

Многие сервисы дают специальный адрес — **API**, — по которому можно получить данные не в виде красивой страницы, а в удобном для программы формате **JSON**. Например, погоду, курсы валют, информацию о книгах.

Библиотека requests

ТЕРМИНАЛ

```
pip install requests
```

PYTHON

```python
import requests

response = requests.get("https://dog.ceo/api/breeds/image/random", timeout=10)
print(response.status_code)
data = response.json()      # превратить JSON-ответ в словарь
print(data)
```

РЕЗУЛЬТАТ

```
200
{'message': 'https://images.dog.ceo/breeds/husky/n02110185_1469.jpg', 'status': 'success'}
```

- `requests.get(адрес)` — отправить запрос и получить ответ;
- `timeout=10` — не ждать ответа дольше 10 секунд;
- `response.status_code` — **код ответа**;
- `response.json()` — содержимое ответа в виде словаря или списка.

Код	Значение
200	всё хорошо
404	не найдено
429	слишком много запросов, подожди
500	ошибка на сервере

Параметры запроса

Часто в запрос нужно передать параметры: город, дату, поисковый запрос. Их передают словарём `params`:

```python
import requests

params = {
    "latitude": 55.75,
    "longitude": 37.62,
    "current": "temperature_2m,wind_speed_10m",
}
response = requests.get("https://api.open-meteo.com/v1/forecast", params=params,
timeout=10)
weather = response.json()["current"]
print(f"Температура: {weather['temperature_2m']} °C")
print(f"Ветер: {weather['wind_speed_10m']} км/ч")
```

РЕЗУЛЬТАТ

```
Температура: 8.4 °C
Ветер: 12.1 км/ч
```

Какие параметры есть у API и что приходит в ответе, написано в его **документации**. Читать документацию — обычная часть работы программиста.

Обработка ошибок сети

Интернет может пропасть, сервер — не ответить. Программа не должна падать:

```python
import requests

try:
    response = requests.get("https://dog.ceo/api/breeds/image/random", timeout=10)
    response.raise_for_status()     # если код ответа — ошибка, создать исключение
    print(response.json()["message"])
except requests.RequestException:
    print("Не удалось получить данные. Проверь интернет")
```

Скачивание файла

Ответ может быть и картинкой. Её содержимое лежит в `response.content`:

```python
import requests

url = requests.get("https://dog.ceo/api/breeds/image/random", timeout=10).json()["message"]
image = requests.get(url, timeout=10)
with open("dog.jpg", "wb") as file:    # "wb" — запись байтов, а не текста
    file.write(image.content)
```

> **Совет.** Открытые API без ключа и регистрации для экспериментов: Open-Meteo (погода), Dog API (картинки собак), REST Countries (страны), Open Library (книги), курсы валют ЦБ РФ (cbr-xml-daily.ru).

8.2 Базы данных: SQLite

JSON-файл хорош для небольших данных. Но когда записей тысячи, по ним нужно искать и считать статистику, а несколько программ работают с данными одновременно, используют **базы данных**.

SQLite — база данных в одном файле. Модуль `sqlite3` уже встроен в Python, ничего устанавливать не нужно.

Таблицы

Данные в базе хранятся в **таблицах**, как в Excel: строка — одна запись, столбец — одно свойство.

id	title	author	year
1	Гарри Поттер	Дж. Роулинг	1997
2	Мастер и Маргарита	М. Булгаков	1967
3	Хоббит	Дж. Толкин	1937

Команды для базы пишутся на специальном языке **SQL**.

Создание таблицы и добавление данных

```python
import sqlite3

connection = sqlite3.connect("library.db")    # файл базы; если его нет — создастся
cursor = connection.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS books (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        author TEXT NOT NULL,
        year INTEGER
    )
""")

cursor.execute(
    "INSERT INTO books (title, author, year) VALUES (?, ?, ?)",
    ("Гарри Поттер", "Дж. Роулинг", 1997),
)
connection.commit()      # сохранить изменения в файл
connection.close()
```

- `CREATE TABLE` — создать таблицу и описать столбцы;
- `PRIMARY KEY AUTOINCREMENT` — уникальный номер записи, проставляется сам;
- `NOT NULL` — поле обязательно;
- `INSERT INTO` — добавить строку;
- `commit()` — сохранить изменения. Без него данные не запишутся!

Параметры ? и SQL-инъекции

Значения в запрос подставляются через знаки `?`, а сами значения передаются отдельно. **Никогда** не вставляй данные пользователя прямо в текст запроса через f-строку:

```python
name = input("Название: ")
cursor.execute(f"SELECT * FROM books WHERE title = '{name}'")    # ОПАСНО!
cursor.execute("SELECT * FROM books WHERE title = ?", (name,))   # правильно
```

Если пользователь введёт специально составленный текст, в первом варианте он сможет изменить запрос — например, удалить всю таблицу. Это называется **SQL-инъекция**. Знаки `?` полностью защищают от неё.

> **Важно.** Если параметр один, его всё равно передают кортежем — с запятой: `(name,)`.

Выборка: SELECT

```python
import sqlite3

connection = sqlite3.connect(":memory:")   # временная база в памяти — для примера
cursor = connection.cursor()
cursor.execute("CREATE TABLE books (id INTEGER PRIMARY KEY, title TEXT, author TEXT, year
INTEGER)")
cursor.executemany(
    "INSERT INTO books (title, author, year) VALUES (?, ?, ?)",
    [
        ("Гарри Поттер", "Дж. Роулинг", 1997),
        ("Мастер и Маргарита", "М. Булгаков", 1967),
        ("Хоббит", "Дж. Толкин", 1937),
        ("Собачье сердце", "М. Булгаков", 1925),
    ],
)

cursor.execute("SELECT title, year FROM books WHERE year > 1950 ORDER BY year")
for title, year in cursor.fetchall():
    print(title, year)

cursor.execute("SELECT author, COUNT(*) FROM books GROUP BY author ORDER BY author")
print(cursor.fetchall())
```

РЕЗУЛЬТАТ

```
Мастер и Маргарита 1967
Гарри Поттер 1997
[('Дж. Роулинг', 1), ('Дж. Толкин', 1), ('М. Булгаков', 2)]
```

SQL	Что делает
`SELECT столбцы FROM таблица`	выбрать данные (`*` — все столбцы)
`WHERE условие`	только строки, где условие верно
`ORDER BY столбец`	сортировка (`DESC` — по убыванию)
`LIMIT 5`	только первые 5 строк
`COUNT(*)`, `SUM()`, `AVG()`	подсчёт, сумма, среднее
`GROUP BY столбец`	сгруппировать строки (например, по автору)
`UPDATE таблица SET ... WHERE ...`	изменить строки
`DELETE FROM таблица WHERE ...`	удалить строки

- `fetchall()` — все строки результата (список кортежей);
- `fetchone()` — одну строку.

> **Важно.** `UPDATE` и `DELETE` без `WHERE` изменят или удалят **все** строки таблицы.

Связи между таблицами

Данные не дублируют. Например, у читателей отдельная таблица, а в таблице выдач хранится только номер читателя. Собрать данные из нескольких таблиц помогает `JOIN`:

```sql
SELECT readers.name, books.title
FROM loans
JOIN readers ON readers.id = loans.reader_id
JOIN books ON books.id = loans.book_id;
```

> **Совет.** Посмотреть, что лежит в базе, удобно в бесплатной программе **DB Browser for SQLite**: там видны таблицы, и запросы можно выполнять мышкой.

8.3 Чат-боты

Бот — это программа, которая общается с пользователями в мессенджере. Боты могут отвечать на вопросы, проводить викторины, присылать напоминания, показывать погоду.

Ниже пример бота для **Telegram** на библиотеке `pyTelegramBotAPI`. Для ВКонтакте есть похожая библиотека `vk_api`, принцип тот же.

Подготовка

1. В Telegram найди бота **@BotFather**, отправь команду `/newbot` и следуй инструкциям.
2. BotFather пришлёт **токен** — длинную строку вида `123456:ABC-DEF...`. Это пароль от твоего бота.
3. Установи библиотеки:

```
ТЕРМИНАЛ
pip install pyTelegramBotAPI python-dotenv
```

Токен — в отдельный файл

Токен нельзя писать прямо в коде: если код попадёт на GitHub, любой сможет управлять твоим ботом. Его кладут в файл `.env`, а файл `.env` добавляют в `.gitignore`.

```
BOT_TOKEN=123456:ABC-DEF...
```

Первый бот

```python
import os
import random

import telebot
from dotenv import load_dotenv

load_dotenv()                                   # прочитать файл .env
bot = telebot.TeleBot(os.getenv("BOT_TOKEN"))

ANSWERS = ["Да!", "Нет.", "Скорее да", "Даже не думай", "Спроси позже"]


@bot.message_handler(commands=["start"])       # команда /start
def start(message):
    bot.send_message(message.chat.id, "Привет! Задай мне вопрос, и я отвечу.")


@bot.message_handler(content_types=["text"])   # любой текст
def answer(message):
    bot.send_message(message.chat.id, random.choice(ANSWERS))


bot.infinity_polling()                          # ждать сообщений бесконечно
```

- `@bot.message_handler(...)` — **декоратор**: «вызывай эту функцию, когда придёт такое сообщение»;
- `message.chat.id` — номер чата, куда отвечать;
- `message.text` — текст сообщения пользователя;
- `message.from_user.first_name` — имя пользователя.

Бот работает, пока запущена программа. Если её закрыть, бот перестанет отвечать.

Кнопки

```python
from telebot import types

@bot.message_handler(commands=["menu"])
def menu(message):
    keyboard = types.ReplyKeyboardMarkup(resize_keyboard=True)
    keyboard.add("🌤 Погода", "🎲 Кубик")
    bot.send_message(message.chat.id, "Выбери:", reply_markup=keyboard)

@bot.message_handler(func=lambda m: m.text == "🎲 Кубик")
def dice(message):
    bot.send_message(message.chat.id, f"Выпало: {random.randint(1, 6)}")
```

Кнопка просто отправляет боту свой текст, а бот реагирует на этот текст.

Хранение данных пользователей

Чтобы бот помнил, например, очки каждого игрока в викторине, данные сохраняют в базе SQLite. Ключом служит `message.from_user.id` — уникальный номер пользователя в мессенджере:

```python
cursor.execute(
    "INSERT OR IGNORE INTO players (user_id, name, score) VALUES (?, ?, 0)",
    (message.from_user.id, message.from_user.first_name),
)
cursor.execute("UPDATE players SET score = score + 1 WHERE user_id = ?",
               (message.from_user.id,))
connection.commit()
```

> **Важно.** Проверь, какой мессенджер доступен на твоём компьютере и в сети, где ты работаешь. Если Telegram не работает, используй бота ВКонтакте — идеи и устройство кода те же.

8.4 Оконные приложения на tkinter ДОП. ТЕМА

tkinter — библиотека для программ с окнами, кнопками и полями ввода. Она встроена в Python.

```python
import tkinter as tk

def convert():
    try:
        celsius = float(entry.get())
        result.config(text=f"{celsius * 9 / 5 + 32:.1f} °F")
    except ValueError:
        result.config(text="Введи число")

window = tk.Tk()
window.title("Конвертер")

tk.Label(window, text="Градусы Цельсия:").pack(padx=10, pady=5)
entry = tk.Entry(window)
entry.pack(padx=10)
tk.Button(window, text="Перевести", command=convert).pack(pady=5)
result = tk.Label(window, text="", font=("Arial", 16))
result.pack(pady=10)

window.mainloop()
```

- `tk.Tk()` — главное окно;
- `Label` — надпись, `Entry` — поле ввода, `Button` — кнопка;
- `command=convert` — какую функцию вызвать при нажатии (без скобок!);
- `pack()` — разместить элемент в окне;
- `entry.get()` — текст из поля ввода;
- `window.mainloop()` — запустить окно и ждать действий пользователя.

ПРИЛОЖЕНИЕ

Шпаргалки

Короткие таблицы, чтобы быстро вспомнить нужное.

Встроенные функции

Функция	Что делает	Пример
`print(...)`	вывести на экран	`print("Привет", end="")`
`input(текст)`	спросить у пользователя (возвращает строку)	`name = input("Имя: ")`
`len(x)`	длина строки, списка, словаря	`len("кот")` → `3`
`type(x)`	тип значения	`type(5)` → `int`
`int()`, `float()`, `str()`, `bool()`	преобразовать тип	`int("7")` → `7`
`list()`, `tuple()`, `set()`, `dict()`	создать коллекцию	`list("abc")` → `['a', 'b', 'c']`
`range(a, b, шаг)`	последовательность чисел	`range(1, 4)` → 1, 2, 3
`sum(x)`, `min(x)`, `max(x)`	сумма, минимум, максимум	`max([3, 9, 2])` → `9`
`abs(x)`	модуль числа	`abs(-5)` → `5`
`round(x, n)`	округлить до `n` знаков	`round(3.14159, 2)` → `3.14`
`sorted(x)`	новый отсортированный список	`sorted([3, 1, 2])` → `[1, 2, 3]`
`reversed(x)`	перебор в обратном порядке	`list(reversed([1, 2, 3]))` → `[3, 2, 1]`
`enumerate(x)`	перебор с номерами	`for i, v in enumerate(lst):`
`zip(a, b)`	перебор двух списков сразу	`for x, y in zip(a, b):`
`any(x)`, `all(x)`	есть ли хоть одно / все ли `True`	`any([False, True])` → `True`
`ord(c)`, `chr(n)`	код символа и символ по коду	`ord("A")` → `65`

Методы строк

Метод	Что делает
`s.upper()`, `s.lower()`	все заглавные / все строчные
`s.strip()`	убрать пробелы по краям
`s.split(разделитель)`	разрезать на список
`разделитель.join(список)`	склеить список в строку
`s.replace(старое, новое)`	заменить подстроку
`s.find(подстрока)`	индекс подстроки или `-1`
`s.count(подстрока)`	сколько раз встречается
`s.startswith(x)`, `s.endswith(x)`	начинается / заканчивается на `x`
`s.isdigit()`, `s.isalpha()`	только цифры / только буквы

Методы списков и словарей

Список	Что делает	Словарь	Что делает
`a.append(x)`	добавить в конец	`d[key] = value`	добавить или изменить
`a.insert(i, x)`	вставить на место `i`	`d.get(key, по_умолч.)`	значение или запасное
`a.pop()`	удалить последний	`d.pop(key)`	удалить ключ
`a.remove(x)`	удалить значение `x`	`d.keys()`	все ключи
`a.sort()`	отсортировать	`d.values()`	все значения
`a.index(x)`	найти индекс	`d.items()`	пары ключ — значение
`a.copy()`	копия списка	`key in d`	есть ли ключ

Частые ошибки

Сообщение	Что значит	Как исправить
`SyntaxError: invalid syntax`	код написан не по правилам	проверь скобки, кавычки, двоеточия
`SyntaxError: expected ':'`	забыто двоеточие	поставь `:` после `if`, `for`, `while`, `def`, `class`
`IndentationError`	неправильный отступ	выровняй отступы (4 пробела)
`NameError: name 'x' is not defined`	переменной нет	проверь опечатку и что переменная создана раньше
`TypeError: can only concatenate str (not "int") to str`	сложение строки и числа	преобразуй: `str(x)` или используй f-строку
`ValueError: invalid literal for int()`	`int()` получил не число	проверь ввод, используй `try/except`
`IndexError: list index out of range`	нет элемента с таким индексом	помни, что индексы с 0, последний — `len - 1`
`KeyError`	нет такого ключа в словаре	проверь ключ или используй `d.get()`
`ZeroDivisionError`	деление на ноль	проверь делитель перед делением
`AttributeError: ... has no attribute`	у объекта нет такого метода	проверь опечатку и тип значения
`ModuleNotFoundError`	модуль не установлен	`pip install имя_модуля`
`FileNotFoundError`	файла нет	проверь имя и папку, из которой запущена программа
`RecursionError`	бесконечная рекурсия	проверь базовый случай

Горячие клавиши VS Code

Клавиши	Что делают
`Ctrl + S`	сохранить файл
`Ctrl + /`	закомментировать / раскомментировать строку
`Ctrl + Z` / `Ctrl + Y`	отменить / вернуть
`Ctrl + D`	выделить следующее такое же слово
`Alt + ↑` / `Alt + ↓`	переместить строку вверх / вниз
`Shift + Alt + ↓`	скопировать строку вниз
`Tab` / `Shift + Tab`	увеличить / уменьшить отступ выделенных строк
`F2`	переименовать переменную везде
`F5`	запустить с отладчиком
`Ctrl + ` `	открыть терминал

Что дальше

Если тебе понравилось программировать, вот куда можно двигаться после этой методички.

Направление	Что изучать
Сайты	Flask или Django — свои сайты на Python
Данные и нейросети	pandas, matplotlib, scikit-learn, основы машинного обучения
Игры	Godot (его язык GDScript очень похож на Python) или Unity на C#
Олимпиады	алгоритмы на графах, динамическое программирование
Автоматизация	скрипты для работы с файлами, таблицами и сайтами

Где учиться и искать ответы

- **docs.python.org/ru** — официальная документация и учебник Python на русском;
- **pythontutor.com** — пошаговое выполнение программ с картинками;
- **stepik.org** — бесплатный курс «Поколение Python» с задачами и автопроверкой;
- **informatics.msk.ru** и **codeforces.com** — задачи для тренировки и олимпиад;
- **visualgo.net** — анимации алгоритмов сортировки и поиска;
- **pyga.me/docs** — документация pygame-ce;
- **kenney.nl** и **opengameart.org** — бесплатная графика и звуки для игр.

> **Совет.** Когда застрял: сначала прочитай сообщение об ошибке, потом поищи ответ сам — 15 минут. Не получилось — спроси, и расскажи, что ты делал, что ожидал и что получил. Так тебе помогут быстрее всего.